



HOCHSCHULE HEILBRONN

Untersuchung unterschiedlicher BiLSTM Netzwerkarchitekturen für Anomalieerkennung

Bachelorarbeit

Zum Erlangen des akademischen Grades eines Bachelor of Engineering

im Studiengang Mechatronik und Robotik
an der Hochschule Heilbronn

von

Julian Krauß

Matrikelnummer: 208765

Betreuer: Prof. Dr.-Ing. Nicolaj Stache & Tobias Schmieg

Bearbeitungszeitraum: 15. Dezember 2024 - 15. April 2025

Abstract

Die Erkennung von Anomalien in Zeitreihendaten ist ein entscheidendes Thema in zahlreichen Anwendungsbereichen, wie der industriellen Überwachung, der Medizindiagnostik und der Finanzanalyse. Diese Arbeit untersucht die Anwendung von rekurrenten neuronalen Netzwerken (RNNs), insbesondere bidirektionalen Long Short-Term Memory Netzwerken (BiLSTM), für die Anomalieerkennung in multivariaten Zeitreihendaten. Dabei werden verschiedene Modellarchitekturen verglichen, einschließlich BiLSTM-Autoencoder (BiLSTM-AE), BiLSTM-Variational Autoencoder (BiLSTM-VAE) und einem Attention-based BiLSTM-Autoencoder (Attention-based BiLSTM-AE). Die Modelle werden auf zwei Datensätzen evaluiert: dem synthetischen Tennessee Eastman Process (TEP) und einem realen industriellen Datensatz der Filling-Station einer Modellfabrik.

Die Analyse der Ergebnisse zeigt, dass die BiLSTM-AE- und Attention-based BiLSTM-AE-Modelle insgesamt die besten Leistungen in Bezug auf die Anomalieerkennung erbringen. Insbesondere das Attention-based Modell konnte sowohl den AUCPR als auch den F1-Score signifikant verbessern, was auf die effektive Nutzung von Attention-Mechanismen hinweist. Trotz ihrer guten Leistung schneiden die Modelle im Vergleich zu anderen fortgeschrittenen Ansätzen, wie denen in der Literatur beschrieben, nicht ganz an der Spitze ab. Dennoch stellen sie eine vielversprechende und ressourcenschonende Lösung für die industrielle Anomalieerkennung dar.

Eigenständigkeitserklärung

Hiermit erkläre ich, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Alle sinngemäß und wörtlich übernommenen Textstellen aus fremden Quellen wurden kenntlich gemacht.

Ort, Datum

Julian Krauß

Inhaltsverzeichnis

1	Einleitung	1
1.1	Einführung und Motivation	1
1.2	Vorgehen und Ziele	2
1.3	Struktur der Arbeit	3
2	Grundlagen	5
2.1	Anomalieerkennung	5
2.2	Evaluationsmetriken in der Anomalieerkennung	7
2.3	Feedforward-Neuronale Netzwerke	10
2.4	Autoencoder	10
2.5	Rekurrente neuronale Netzwerke	13
2.6	Long Short-Term Memory	14
2.7	Attention Heads	18
2.8	Modellfabrik	21
3	Verwandte Arbeiten	23
3.1	BiLSTM Autoencoder	23
3.2	BiLSTM Variational Autoencoder	23
3.3	Attention-based BiLSTM Autoencoder	24
4	Daten	25
4.1	Datenaufnahme an der Modellfabrik	25
4.2	Datenvorverarbeitung	25
4.3	TEP-Datensatz	29
4.4	Filling-Station-Datensatz	32
5	Methodik	37
5.1	Anomalieerkennung	37
5.2	Modelle	39
5.3	Training und Evaluierung	43
6	Ergebnisse	47
6.1	TEP-Datensatz	47
6.2	Filling-Station-Datensatz	51
7	Diskussion	55
7.1	TEP-Datensatz	55
7.2	Filling-Station-Datensatz	59
7.3	Zusammenführung der Diskussionsergebnisse	60
8	Zusammenfassung und Ausblick	61
8.1	Zusammenfassung	61
8.2	Kritische Bewertung der Ergebnisse	62
8.3	Ausblick	63
	Literatur	65

Abbildungsverzeichnis

2.1	Zeitreihen Rekonstruktion	6
2.2	Zeitreihen Vorhersage [16]	6
2.3	Rekonstruktion der Zeitreihe mit Anomalie und zugehörigem Anomalie-Score	7
2.4	Beispielhafte AUCPR-Kurve	9
2.5	Architektur einer LSTM-Zelle [28]	16
2.6	Architektur eines bidirektionalen LSTM-Netzwerks mit Sigmoid Aktivierungsfunktion [30]	18
2.7	Modellfabrik der HHN im Otto Rettenmaier Forschungslabor [7]	21
2.8	Filling-Station der Modellfabrik [7]	22
4.1	Illustration des Sliding Window-Verfahrens [37]	27
4.2	Korrelationsmatrix der relevantesten Variablen im TEP-Datensatz. Starke Korrelationen sind in dunklen Farben markiert.	30
4.3	Normale und Anomale Daten der Variable xmeas_1	31
4.4	Motorstrom am Drehtisch im Normalzustand [7]	34
4.5	Drehtisch mit Widerstand [7]	34
4.6	Drehvorgang mit zu geringem Widerstand [7]	34
4.7	Motorstrom am Förderband im Normalzustand [7]	35
4.8	Motorstrom am Förderband mit Widerstand [7]	35
5.1	Schematische Darstellung des BiLSTM-AE Modells [45]	40
5.2	Schematische Darstellung des BiLSTM-VAE-Modells [46]	41
5.3	Architektur eines Multi-Head Attention-based BiLSTM-AE [50]	43
5.4	Verlauf der Lernratenkurve	44
6.1	Originalzeitreihe im Normalzustand mit zugehöriger Rekonstruktion	47
6.2	Originalzeitreihe im anomalen Zustand mit zugehöriger Rekonstruktion und Anomalie-Score	47
6.3	Boxplot des AUCPR Wertes des TEP-Datensatzes	50
6.4	Boxplot des F1-Score-Wertes des TEP-Datensatzes	50
6.5	Modellrekonstruktion und Anomalie-Score der Variable Motorstrom am Drehtisch ohne Anomalie	51
6.6	Modellrekonstruktion und Anomalie-Score der Variable Motorstrom am Drehtisch mit der Anomalie „zu wenig Widerstand“	51

6.7	Modellrekonstruktion und Anomalie-Score der Variable Motorstrom am Drehtisch mit der Anomalie „zu viel Widerstand“	51
6.8	Boxplot des AUCPR Wertes des Filling-Station-Datensatzes	54
6.9	Boxplot des F1-Score-Wertes des Filling-Station-Datensatzes	54

Tabellenverzeichnis

2.1	Sensor- und Steuervariablen der Filling-Station [7]	22
4.1	Korrelationen zwischen Variablen im TEP-Datensatz	31
4.2	Parameter der Datenvorverarbeitung des TEP-Datensatzes	32
4.3	Parameter der Datenverarbeitung des Filling-Station-Datensatzes	35
5.1	Parameter für das BiLSTM-AE Modell mit TEP-Datensatz	45
5.2	Parameter für das BiLSTM-AE Modell mit Filling-Station-Datensatz	45
5.3	Parameter für das BiLSTM-VAE Modell mit TEP-Datensatz	46
5.4	Parameter für das BiLSTM-VAE Modell mit Filling-Station-Datensatz	46
5.5	Parameter für das Attention-based BiLSTM-AE Modell mit TEP-Datensatz	46
5.6	Parameter für das Attention-based BiLSTM-AE Modell mit Filling-Station-Datensatz	46
6.1	Ergebnisse der Modelle, die auf dem TEP-Datensatz trainiert wurden, mit den Metriken: AUCPR, F1-Score, Precision und Recall	49
6.2	Ergebnisse der Modelle, die auf dem TEP-Datensatz trainiert wurden, mit den Metriken: Accuracy, MCC und Trainingsdauer	49
6.3	Ergebnisse der Modelle, die auf dem Filling-Station-Datensatz trainiert wurden, mit den Metriken: AUCPR, F1-Score, Precision und Recall	53
6.4	Ergebnisse der Modelle, die auf dem Filling-Station-Datensatz trainiert wurden, mit den Metriken: Accuracy, MCC und Trainingsdauer	53
7.1	Vergleich der besten Modelle aus dieser Arbeit mit den Ergebnissen aus [51] auf Basis der Metriken AUCPR und F1-Score	58

Abkürzungsverzeichnis

ML Machine Learning

DL Deep Learning

HMM Hidden Markov Model

RNN Rekurrentes-Neuronales Netzwerk

LSTM Long Short-Term Memory

BiLSTM Bidirektionale Long Short-Term Memory Netzwerke

FNN Feedforward-Neuronales Netzwerk

AE Autoencoder

VAE Variational Autoencoder

AUCPR Area Under Precision-Recall

AUCROC Area Under Receiver Operating Characteristic

TEP Tennessee Eastman Proces

MCC Matthews Correlation Coefficient

1 Einleitung

1.1 Einführung und Motivation

Die Erkennung von Anomalien in Zeitreihen ist eine zentrale Herausforderung in verschiedenen Anwendungsbereichen wie der industriellen Fertigung, der Finanzanalyse oder der Medizindiagnostik [1]. Anomalien können auf kritische Ereignisse wie Maschinenausfälle, Sicherheitsverletzungen oder ungewöhnliche Muster in Sensordaten hinweisen. Die präzise und schnelle Identifikation solcher Abweichungen ist entscheidend, um ungeplante Stillstände zu minimieren, Kosten zu senken und die Sicherheit zu gewährleisten [2]. Moderne Produktionsanlagen sind zunehmend mit Sensoren ausgestattet, die eine kontinuierliche Überwachung ermöglichen. Durch das industrielle Internet der Dinge entstehen große Mengen an multivariaten (multidimensionalen) Zeitreihendaten, deren manuelle Analyse jedoch kaum praktikabel ist [3]. Maschinelles Lernen bietet hier einen vielversprechenden Ansatz zur Automatisierung der Anomalieerkennung.

In den letzten Jahren haben tiefe neuronale Netze und insbesondere rekurrente Architekturen wie Long Short-Term Memory (LSTM) Netzwerke große Fortschritte in der Modellierung von Zeitreihendaten erzielt [4]. Dabei haben sich Autoencoder-Modelle als effektive Methode zur Erkennung von Anomalien erwiesen, indem sie versuchen, die zugrunde liegende Struktur der Daten zu rekonstruieren und Abweichungen als Anomalien zu klassifizieren [5]. Neben klassischen LSTM-basierten Autoencodern wurden Variational Autoencoder (VAE) eingeführt, die eine probabilistische Modellierung der latenten Raumdarstellung ermöglichen und eine verbesserte Generalisierung bieten [5].

Ein weiterer vielversprechender Ansatz zur Verarbeitung sequentieller Daten ist der Attention-Mechanismus, der durch die Transformer-Architektur bekannt wurde [6]. Während Transformer-Modelle im Bereich der natürlichen Sprachverarbeitung bahnbrechende Erfolge erzielten, werden sie zunehmend auch in der Zeitreihenanalyse eingesetzt, um langreichweitige Abhängigkeiten effizient zu modellieren [1].

Diese Arbeit untersucht die Anomalieerkennung in Zeitreihendaten unter Verwendung dreier verschiedener Modelltypen: BiLSTM-Autoencoder, BiLSTM-Variational Autoencoder und ein BiLSTM-Modell mit Attention-Mechanismus. Dabei wird analysiert, inwiefern die Kombination von rekurrenten Netzwerken mit probabilistischen und

aufmerksamkeitsbasierten Ansätzen die Genauigkeit der Anomalieerkennung verbessern kann. Durch den Vergleich dieser Modelle wird ein umfassender Überblick über die Leistungsfähigkeit unterschiedlicher Architekturen für die industrielle Anomalieerkennung gegeben [7].

1.2 Vorgehen und Ziele

Das Ziel dieser Arbeit ist die Entwicklung und Evaluierung eines Ansatzes zur Anomalieerkennung in multivariaten Zeitreihendaten unter Verwendung rekurrenter neuronaler Netze. Im Fokus stehen, wie schon erwähnt, die drei Modelltypen: BiLSTM-Autoencoder, BiLSTM-Variational Autoencoder sowie BiLSTM-Modell mit Attention-Mechanismus. Ziel dieser Untersuchung ist es, die Stärken und Schwächen dieser Ansätze hinsichtlich ihrer Rekonstruktionsgenauigkeit und ihrer Fähigkeit zur Identifikation von Anomalien zu analysieren [1], [5].

Um dieses Ziel zu erreichen, werden zunächst die theoretischen Grundlagen der Autoencoder-basierten Anomalieerkennung erarbeitet. Anschließend wird ein experimenteller Rahmen entwickelt, in dem die Modelle trainiert und validiert werden. Die Architektur der Modelle wird so gewählt, dass sowohl die zeitlichen Abhängigkeiten in den Daten als auch komplexe Zusammenhänge innerhalb der multivariaten Zeitreihen optimal erfasst werden. Dabei spielen Aspekte wie die Auswahl der Hyperparameter, die Regularisierung sowie die Skalierung der Eingabedaten eine entscheidende Rolle [2].

Zur Evaluierung der entwickelten Modelle werden zwei Datensätze verwendet. Zunächst erfolgt eine Validierung auf einem öffentlich verfügbaren Benchmark-Datensatz, um eine Vergleichbarkeit mit bestehenden Arbeiten herzustellen. Dies ermöglicht eine erste Einschätzung der Leistungsfähigkeit der Ansätze im Vergleich zu etablierten Methoden. Darüber hinaus wird ein weiterer Datensatz aus einer realen industriellen Umgebung analysiert. Dieser enthält Sensordaten, die typische Betriebszustände sowie anomale Zustände einer industriellen Produktionsanlage widerspiegeln. Durch die Anwendung der Modelle auf reale Daten kann die Übertragbarkeit der entwickelten Methoden auf praxisrelevante Szenarien überprüft werden.

Die Ergebnisse dieser Arbeit sollen zeigen, welche der untersuchten Modellarchitekturen sich am besten für die Anomalieerkennung in multivariaten Zeitreihen eignen. Durch

die detaillierte Analyse der Modellleistung auf synthetischen und realen Daten soll ein umfassendes Verständnis für die Vorteile und Herausforderungen der verschiedenen Ansätze gewonnen werden.

1.3 Struktur der Arbeit

Der Aufbau dieser Arbeit gliedert sich in mehrere Abschnitte, beginnend mit den theoretischen Grundlagen in Abschnitt 2. Hier werden die wesentlichen Konzepte der Anomalieerkennung in Zeitreihen vorgestellt, darunter verschiedene Evaluationsmetriken sowie eine detaillierte Beschreibung der verwendeten rekurrenten neuronalen Netze. Diese theoretischen Grundlagen bilden die Basis für die nachfolgenden Untersuchungen.

In Abschnitt 3 werden bestehende Publikationen zu neuronalen Netzwerken in der Anomalieerkennung analysiert. Dabei liegt der Fokus auf bisherigen Arbeiten, die sich mit rekurrenten neuronalen Netzwerken für die Modellierung von Zeitreihenanomalien beschäftigen.

Die in dieser Arbeit verwendeten Datensätze werden in Abschnitt 4 behandelt. Zunächst wird der synthetische Benchmark-Datensatz Tennessee Eastman Process (TEP) beschrieben, der häufig zur Evaluierung von Anomalieerkennungsverfahren verwendet wird. Anschließend wird ein Datensatz vorgestellt, der aus einer realen industriellen Umgebung stammt. Der Datensatz enthält Sensordaten einer Filling-Station, wobei sowohl der Normalzustand als auch anomale Sequenzen enthalten sind, die für das Training und die Evaluierung der Modelle verwendet werden.

In Abschnitt 5 wird die Methodik dieser Arbeit erläutert. Zunächst wird die gewählte Strategie zur Anomalieerkennung beschrieben, gefolgt von einer detaillierten Darstellung der implementierten Modellarchitekturen. Dabei wird auf die spezifischen Eigenschaften der BiLSTM-AE-, BiLSTM-VAE- und BiLSTM-Attention-Modelle eingegangen. Darüber hinaus wird der Trainingsprozess erläutert, einschließlich der Wahl der Hyperparameter, der Optimierungsmethoden sowie der Evaluierungsstrategien.

Die Resultate der durchgeführten Experimente werden in Abschnitt 6 dargestellt. Die Modelle BiLSTM-AE, BiLSTM-VAE und BiLSTM-Attention werden hinsichtlich ihrer Rekonstruktionsleistung und Anomalieerkennungsfähigkeit miteinander verglichen. Zur Bewertung der Modelle werden verschiedene Evaluationsmetriken herangezogen, darunter

AUCPR und F1-Score. Zusätzlich werden beispielhafte rekonstruierte Zeitreihen präsentiert, um visuelle Unterschiede zwischen den Modellen zu veranschaulichen.

Abschnitt 7 enthält die Analyse und Interpretation der in den vorherigen Kapiteln präsentierten Ergebnisse. Die Stärken und Schwächen der einzelnen Modellarchitekturen werden herausgearbeitet und mögliche Verbesserungsansätze aufgezeigt.

2 Grundlagen

In folgendem Kapitel werden die benötigten Grundlagen zum Verständnis dieser Arbeit im Bereich der Anomalieerkennung mit Deep Learning (DL) Methoden in Verbindung mit Zeitreihen erläutert. Darüber hinaus wird die Modellfabrik, an der die realen Daten aufgenommen wurden, beschrieben.

2.1 Anomalieerkennung

Die Erkennung von Anomalien in Zeitreihen ist ein essenzielles Forschungsfeld mit Anwendungen in zahlreichen Bereichen, darunter industrielle Überwachung, Finanzwesen, Medizindiagnostik und Cybersicherheit [8], [9]. Anomalien sind ungewöhnliche Muster oder Abweichungen in Daten, die auf seltene, unerwartete oder potenziell kritische Ereignisse hinweisen. Solche Abweichungen können auf Systemausfälle, Produktionsfehler oder gesundheitliche Anomalien wie Herzrhythmusstörungen hindeuten [10].

Traditionell wurden statistische Verfahren wie z-Score-Analyse, Hauptkomponentenanalyse (PCA) und Hidden Markov Models (HMMs) zur Anomalieerkennung eingesetzt [11], [12]. Mit der steigenden Verfügbarkeit großer und hochdimensionaler Datenmengen haben sich jedoch Methoden des maschinellen Lernens (ML) und insbesondere des Deep Learning (DL) als leistungsfähige Alternativen etabliert [1]. DL-Methoden ermöglichen es, komplexe nichtlineare Zusammenhänge in den Daten zu erfassen, die mit klassischen Verfahren schwer zu modellieren sind.

Im Allgemeinen lassen sich Anomalieerkennungsverfahren in drei Kategorien einteilen: Supervised Learning, Unsupervised Learning und Semi-Supervised Learning [2]. Beim Supervised Learning wird ein Modell mit gelabelten Daten trainiert, wobei sowohl normale als auch anomale Instanzen bekannt sind. Dieser Ansatz erfordert jedoch eine große Menge an Anomalie-Labels, die in vielen Anwendungsfällen schwer zu beschaffen sind. Beim Unsupervised Learning hingegen kennt das Modell keine expliziten Labels und versucht, Anomalien durch statistische Abweichungen oder Cluster-Analyse zu identifizieren [13]. Semi-Supervised Learning stellt einen Mittelweg dar, indem es das Modell darauf trainiert, ausschließlich den Normalzustand zu lernen. Anomalien werden dann als starke Abweichungen von der gelernten Normalität erkannt [1].

Insbesondere für zeitabhängige Daten gibt es zwei zentrale Methoden der Deep-Learning-basierten Anomalieerkennung: die Rekonstruktion und die Zeitreihenvorhersage. Bei der Rekonstruktion wird ein Modell, beispielsweise ein Autoencoder, trainiert, um Eingabedaten möglichst genau zu rekonstruieren. Der Rekonstruktionsfehler wird dann als Anomalie-Score interpretiert [14]. In Abbildung 2.1 ist eine Zeitreihenrekonstruktion dargestellt. Die blaue Kurve ist die tatsächliche Kurve, während die rote Kurve eine Rekonstruktion darstellt.

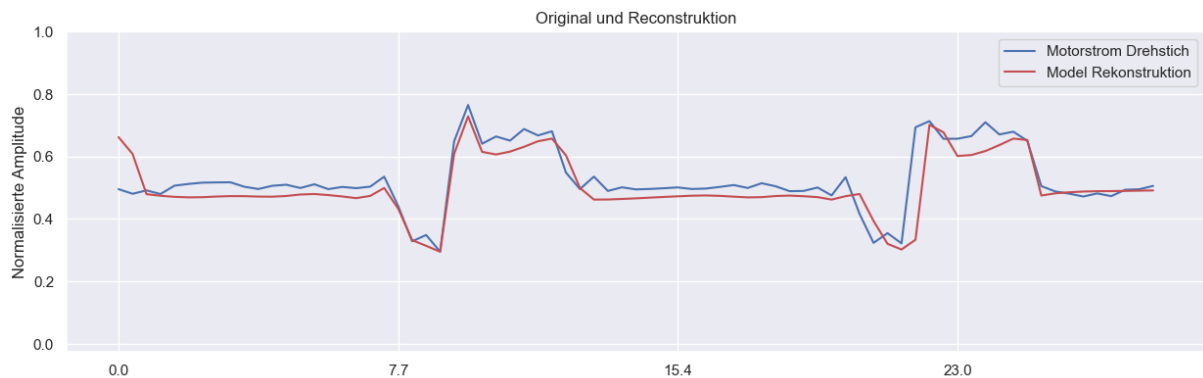


Abbildung 2.1: Zeitreihen Rekonstruktion

Bei der Zeitreihenvorhersage wird das Modell darauf trainiert, zukünftige Werte einer Sequenz vorherzusagen. Wenn eine große Abweichung zwischen der Vorhersage und dem tatsächlichen Wert auftritt, deutet dies auf eine mögliche Anomalie hin [15]. In Abbildung 2.2 ist eine Zeitreihenvorhersage dargestellt. Die schwarze Kurve ist die tatsächliche Kurve, während die blaue Kurve ab dem gewünschten Zeitpunkt die Vorhersage darstellt.

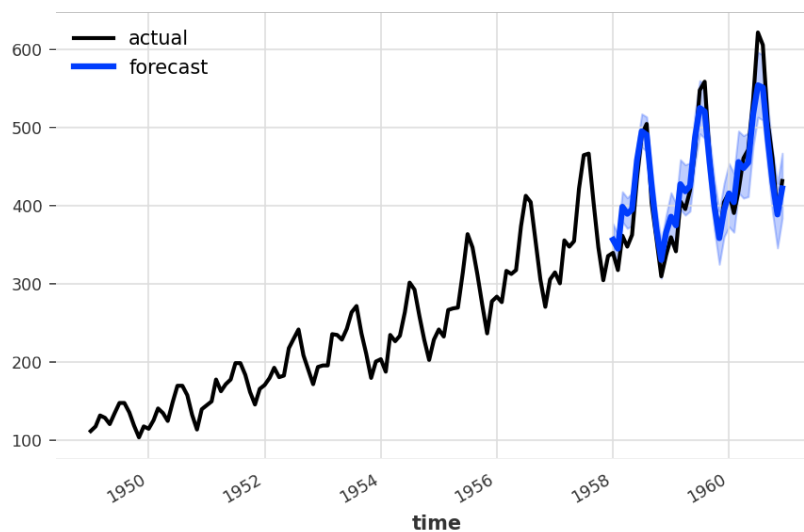


Abbildung 2.2: Zeitreihen Vorhersage [16]

In Abbildung 2.3 ist zur Veranschaulichung des Anomalie-Scores ein Plot einer rekonstruierten Zeitreihe und der Originalzeitreihe mit einer erkennbaren anomalen Sequenz dargestellt. Die obere Darstellung zeigt die ursprünglichen Sensordaten (blaue Linie) im Vergleich zur durch das Modell rekonstruierten Zeitreihe (rote Linie). Im Idealfall sollten die rekonstruierten Werte für normale Zustände der Originalzeitreihe sehr ähnlich sein, während bei Vorliegen einer Anomalie deutliche Abweichungen auftreten.

Die untere Darstellung visualisiert den Anomalie-Score, welcher als Maß für die Abweichung zwischen Original und Rekonstruktion dient. Ein erhöhter Anomalie-Score (orange Linie) zeigt Zeiten an, in denen das Modell größere Differenzen zwischen Eingangs- und Ausgangssignal festgestellt hat, was auf das Vorhandensein von Anomalien hinweist.

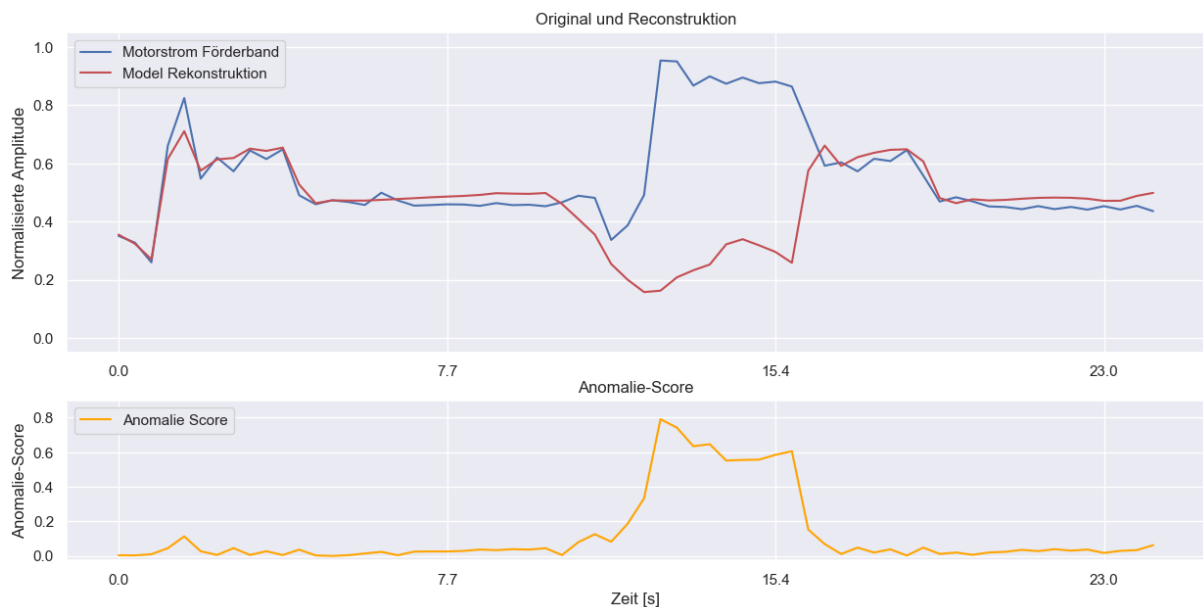


Abbildung 2.3: Rekonstruktion der Zeitreihe mit Anomalie und zugehörigem Anomalie-Score

2.2 Evaluationsmetriken in der Anomalieerkennung

Um die Leistung eines Modells zu bewerten und um Modelle miteinander zu vergleichen, werden Bewertungsmetriken benötigt. Übliche Metriken für die Erkennung von Anomalien in Zeitreihen sind der F1-Score, die Area Under Precision-Recall Curve (AUCPR) und die Area Under Receiver Operating Characteristic Curve (AUCROC).

Da der Anomalie-Score ein stetiger Wert und die Labels ein boolescher Wert sind, können diese nicht direkt miteinander verglichen werden. Um den Anomalie-Score in einen booleschen Wert umzuwandeln, muss ein Schwellenwert definiert werden. Wird

dieser Schwellenwert überschritten, wird für diese Position eine Anomalie markiert. Dieser Schwellenwert ist jedoch schwierig zu definieren und variiert je nach Anwendung, Modell und kann sogar von Training zu Training unterschiedlich sein.

Um eine einheitliche und vergleichbare Metrik zu erhalten, wird häufig der F1-Best verwendet, wobei der F1-Score (siehe 2.1) für alle möglichen Schwellenwerte berechnet und der beste Wert ausgewählt wird. Allerdings gibt es auch hier Herausforderungen, da der F1-Score auf verschiedene Weise durch Mittelwertbildung der Klassenhäufigkeiten berechnet werden kann. Dies ist insbesondere dann relevant, wenn die Klassen ungleich verteilt sind, wie es typischerweise bei der Anomalieerkennung der Fall ist [1], [2].

$$F1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (2.1)$$

Die Metriken AUC-ROC und AUCPR sind ähnlich, wobei AUCPR in der Anomalieerkennung in der Literatur häufiger verwendet wird. Beide Metriken berechnen die Fläche unter einer Kurve, wobei die Kurve durch die Berechnung zweier Werte für jeden möglichen Schwellenwert entsteht. Im Falle von AUCPR sind dies die Precision (Gleichung 2.2) und der Recall (Gleichung 2.3):

$$\text{Precision} = \frac{\text{True Positive}}{\text{True Positive} + \text{False Positive}} \quad (2.2)$$

$$\text{Recall} = \frac{\text{True Positive}}{\text{True Positive} + \text{False Negative}} \quad (2.3)$$

Je größer die Fläche unter der Kurve ist, desto besser ist das Ergebnis. Ein Beispiel für eine Precision-Recall-Kurve ist in Abb. 2.4 dargestellt.

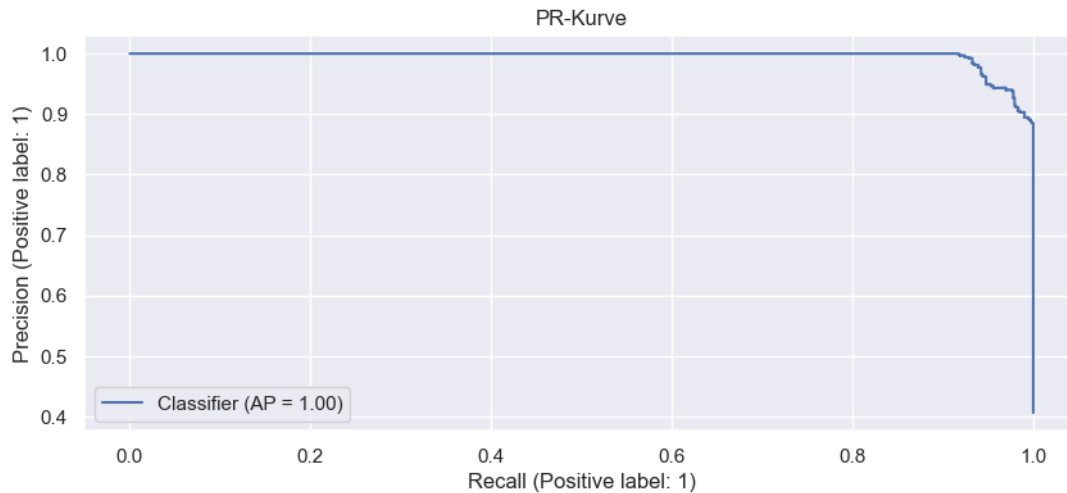


Abbildung 2.4: Beispielhafte AUCPR-Kurve

Wenn der Schwellenwert 0 ist, wird alles als Anomalie erkannt. Da keine Anomalie fälschlicherweise als normal erkannt werden kann, gilt False Negative = 0 und somit Recall = 1.

Wenn sich der Schwellenwert dem Maximum nähert, wird immer weniger als Anomalie erkannt. Dadurch werden keine Normalzustände fälschlicherweise als Anomalien erkannt (False Positive = 0), sodass Precision = 1 ist. Gleichzeitig läuft der Recall jedoch gegen 0, da False Negative immer größer wird.

Wenn das Modell „perfekt“ ist, gibt es für bestimmte Schwellenwerte keine False Positive oder False Negative Erkennungen. In diesem Fall befinden sich alle Punkte der Kurve bei Precision = 1 und Recall = 1, sodass die Fläche unter der Kurve den Wert 1 erreicht.

Die Matthews Correlation Coefficient (MCC) ist eine weitere wichtige Metrik zur Bewertung der Modellleistung, besonders bei unausgewogenen Klassen, wie sie in der Anomalieerkennung häufig vorkommen. Im Gegensatz zum F1-Score oder AUCPR, die sich nur auf die positive Klasse konzentrieren, berücksichtigt der MCC alle vier möglichen Klassifikationen: True Positives, False Positives, True Negatives und False Negatives.

Der MCC-Wert liegt zwischen -1 und $+1$, wobei $+1$ eine perfekte Vorhersage und 0 eine zufällige Klassifikation bedeutet. Ein höherer MCC-Wert zeigt eine bessere Leistung des Modells [17].

Die Berechnung des MCC erfolgt durch die folgende Formel:

$$\text{MCC} = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \quad (2.4)$$

(TP = True Positives, TN = True Negatives, FP = False Positives, FN = False Negatives)

2.3 Feedforward-Neuronale Netzwerke

Feedforward-Neuronale Netzwerke (FNNs) gehören zu den grundlegenden Architekturen im Bereich des maschinellen Lernens. Sie bestehen aus mehreren Schichten künstlicher Neuronen, die die Eingaben schrittweise verarbeiten. Informationen fließen dabei nur in eine Richtung – von der Eingabeschicht über eine oder mehrere versteckte Schichten zur Ausgabeschicht – ohne rekurrente oder zyklische Verbindungen [18]. Jedes Neuron bildet zunächst eine gewichtete Summe seiner Eingabesignale, verarbeitet diese mit einer Aktivierungsfunktion und leitet das resultierende Ergebnis an die nächste Schicht weiter.

FNNs werden häufig für Klassifikations- und Regressionsprobleme eingesetzt, bei denen die Netzwerkausgabe eine Vorhersage über eine Zielvariable trifft. Ihre Architektur kann jedoch auch für andere Aufgaben genutzt werden, beispielsweise für die Rekonstruktion von Daten. Ein spezielles Beispiel hierfür ist der Autoencoder.

2.4 Autoencoder

Autoencoder sind eine spezielle Art neuronaler Netzwerke, die darauf ausgelegt sind, Daten in einem komprimierten Format zu repräsentieren, indem sie die wesentlichen Merkmale der Eingaben lernen und diese für Rekonstruktions- oder Anomalieerkennungsaufgaben nutzen. Im folgenden werden zwei Arten von Autoencodern näher erläutert.

2.4.1 Klassischer Autoencoder

Ein Autoencoder ist ein neuronales Netzwerk, das darauf trainiert wird, seine Eingaben so genau wie möglich zu rekonstruieren. Dabei besteht die Architektur aus zwei Hauptkomponenten: einem Encoder und einem Decoder. Der Encoder reduziert die Eingabedaten auf eine latente Repräsentation, die oft eine niedrigere Dimension als die ursprünglichen Daten hat. Der Decoder versucht anschließend, die ursprünglichen Eingabedaten aus dieser latenten Repräsentation wiederherzustellen [19].

Formal betrachtet nimmt der Encoder eine Eingabe x und wandelt sie in eine latente Darstellung z um:

$$z = f_{\theta}(x) \quad (2.5)$$

wobei f_{θ} eine nicht-lineare Funktion ist, die durch ein neuronales Netzwerk parametrisiert wird. Der Decoder rekonstruiert anschließend die ursprünglichen Daten durch eine Funktion g_{ϕ} , sodass:

$$\hat{x} = g_{\phi}(z) \quad (2.6)$$

Das Ziel des Autoencoders besteht darin, den Rekonstruktionsfehler zu minimieren, sodass die rekonstruierten Daten $\hat{x}$ möglichst nahe an den Eingabedaten x liegen. Dies geschieht durch Optimierung einer Verlustfunktion, häufig der mittleren quadratischen Abweichung (Mean Squared Error, MSE) [20]:

$$L(x, \hat{x}) = \|x - \hat{x}\|^2 \quad (2.7)$$

Ein wesentlicher Aspekt von Autoencodern ist ihre Fähigkeit zur Dimensionsreduktion. Durch die Reduktion auf eine latente Variable z kann das Modell lernen, wichtige Merkmale der Daten zu extrahieren. Daher werden Autoencoder häufig zur Merkmalsextraktion, zur Bildkompression und zur Anomalieerkennung eingesetzt [21].

2.4.2 Variational Autoencoder

Ein Variational Autoencoder (VAE) ist ein generatives Modell, das eine probabilistische Variante des klassischen Autoencoders darstellt. Es wurde von Kingma und Welling eingeführt und basiert auf der Variational Bayesian Inference. Der VAE nutzt eine stochastische latente Repräsentation, indem er anstelle einer festen Kodierung Wahrscheinlichkeitsverteilungen modelliert. Dies ermöglicht die gezielte Generierung neuer Datenpunkte, indem man aus der latenten Verteilung sampelt [22], [23].

Kernprinzipien eines VAE

Der Encoder, auch als Inferenzmodell bezeichnet, approximiert die latente Verteilung $q_{\theta}(z|x)$, indem er eine Wahrscheinlichkeitsverteilung mit einem Mittelwert μ und einer Standardabweichung σ berechnet. Dadurch wird anstelle einer festen Punktkodierung eine flexiblere Darstellung ermöglicht. Um das Sampling während des Trainings differenzierbar

zu machen, wird der sogenannte Reparametrization Trick verwendet. Dabei wird eine zufällige Variable $\epsilon \sim \mathcal{N}(0, 1)$ eingeführt und die latente Variable als

$$z = \mu + \sigma \cdot \epsilon \quad (2.8)$$

definiert.

Nachdem die latente Variable bestimmt wurde, nutzt der Decoder die Verteilung $p_\phi(x|z)$, um die ursprüngliche Eingabe zu rekonstruieren. Ziel ist es, den Decoder so zu trainieren, dass er aus der latenten Repräsentation eine möglichst realistische Rekonstruktion der ursprünglichen Daten erzeugt.

Eine zentrale Komponente des VAE ist die Zielfunktion, die aus zwei Bestandteilen besteht. Zum einen gibt es einen Rekonstruktionsverlust, der meist durch eine mittlere quadratische Abweichung (Mean Square Error, MSE) oder eine Kreuzentropie-Funktion (Cross-Entropy Loss, **CE!**) berechnet wird. Zum anderen wird ein Regularisierungsterm in Form der Kullback-Leibler-Divergenz zwischen der Posterior-Verteilung $q_\theta(z|x)$ und der a-priori-Verteilung $p(z)$ hinzugefügt. Diese wird in der Regel als Standard-Normalverteilung $\mathcal{N}(0, 1)$ gewählt, um eine regulierte latente Struktur zu gewährleisten.

Zielfunktion

Die gesamte Optimierung erfolgt durch Maximierung der Evidence Lower Bound (ELBO), die wie folgt definiert ist:

$$L(x|\theta, \phi) = -KL(q_\theta(z|x)||p(z)) + \mathbb{E}_{q_\theta(z|x)} \log p_\phi(x|z). \quad (2.9)$$

Hierbei beschreibt der erste Term die Kullback-Leibler-Divergenz zwischen der a-priori- und der a-posteriori-Verteilung, während der zweite Term die Wahrscheinlichkeit der Rekonstruktion des Eingabedatums durch das Modell maximiert.

2.4.3 Unterschiede in der Struktur

Ein klassischer Autoencoder rekonstruiert Daten durch eine deterministische Abbildung der Eingabe in eine latente Repräsentation. Die latenten Variablen z sind feste Punkte im latenten Raum, die direkt aus der Eingabe x berechnet werden.

Im Gegensatz dazu verfolgt der Variational Autoencoder (VAE) einen probabilistischen Ansatz. Anstatt eine feste Punktkodierung zu lernen, wird die latente Variable z als eine Wahrscheinlichkeitsverteilung $q_\theta(z|x)$ modelliert. Diese Verteilung wird durch einen Mittelwert μ und eine Standardabweichung σ parametrisiert. Während des Trainings wird die latente Variable nicht direkt aus der Eingabe bestimmt, sondern durch Sampling aus dieser Verteilung erzeugt. Der sogenannte Reparametrisierungs-Trick stellt sicher, dass dieses Sampling differenzierbar bleibt.

Ein weiterer wesentlicher Unterschied liegt in der Zielfunktion. Während der klassische Autoencoder nur den Rekonstruktionsfehler minimiert, optimiert der VAE zusätzlich die Kullback-Leibler-Divergenz zwischen der approximierten latenten Verteilung und einer vorher festgelegten a-priori-Verteilung, meist einer Standard-Normalverteilung $\mathcal{N}(0, 1)$. Dadurch wird die Struktur des latenten Raums reguliert, sodass ein kontinuierlicher und glatter Raum entsteht, in dem ähnliche Eingaben nahe beieinander liegen.

2.5 Rekurrente neuronale Netzwerke

Rekurrente neuronale Netzwerke (RNNs) sind eine spezielle Klasse neuronaler Netze, die darauf ausgelegt sind, sequenzielle Daten zu verarbeiten, indem sie Informationen aus früheren Zeitschritten beibehalten [24]. Im Gegensatz zu Feedforward-Netzwerken, bei denen der Informationsfluss nur in eine Richtung erfolgt, besitzen RNNs rekurrente Verbindungen, die es ermöglichen, vergangene Zustände für die Verarbeitung aktueller Eingaben zu nutzen.

Die zentrale Idee eines RNNs besteht darin, zu jedem Zeitpunkt t einen verborgenen Zustand h_t zu berechnen, der sowohl die aktuelle Eingabe x_t als auch den vorherigen Zustand h_{t-1} berücksichtigt:

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b). \quad (2.10)$$

Dabei sind W_h und W_x Gewichtsmatrizen, die den Einfluss des vorherigen Zustands und der aktuellen Eingabe bestimmen. b ist ein Bias-Term. Die Aktivierungsfunktion $\tanh$ sorgt für eine nicht lineare Transformation des Zustands.

RNNs werden insbesondere in Aufgaben verwendet, bei denen die Reihenfolge der Daten eine entscheidende Rolle spielt, wie z. B. in der Sprachverarbeitung oder der

Zeitreihenanalyse [25]. Allerdings sind klassische RNNs aufgrund des Problems des verschwindenden und explodierenden Gradienten (Vanishing and exploding Gradient) in ihrer Fähigkeit begrenzt, langfristige Abhängigkeiten zu modellieren [26]. Dies bedeutet, dass der Einfluss früherer Zeitschritte mit zunehmender Sequenzlänge exponentiell abnimmt oder instabil wird, was das Training erschwert.

Zur Lösung dieses Problems wurden erweiterte Architekturen wie das Long Short-Term Memory (LSTM) Netzwerk entwickelt, das durch spezialisierte Speichermechanismen eine effizientere Langzeitabhängigkeit ermöglicht [27].

2.6 Long Short-Term Memory

Während herkömmliche RNNs Schwierigkeiten haben, Abhängigkeiten über lange Zeiträume hinweg zu erfassen, ermöglichen es LSTM-Netzwerke, Informationen über viele Zeitschritte hinweg zu speichern und selektiv abzurufen. Dies geschieht durch eine gezielte Steuerung des Informationsflusses innerhalb einer LSTM-Zelle.

Jede LSTM-Zelle besteht aus einem Zellzustand C_t , der als Speicher fungiert, und einem verborgenen Zustand h_t , der als Zwischenausgabe dient. Um den Informationsfluss zu steuern, besitzt jede Zelle drei regulierende Mechanismen: das Vergessensgate (Forget Gate), das Eingangsgate (Input Gate) und das Ausgangsgate (Output Gate). Die Aktivierung dieser Gates basiert auf gewichteten Kombinationen des vorherigen verborgenen Zustands h_{t-1} und der aktuellen Eingabe x_t , wobei *Sigmoid* und *Tanh* als Aktivierungsfunktionen zum Einsatz kommen.

Das Vergessensgate f_t entscheidet, welche Informationen aus dem Zellzustand entfernt werden sollen. Es wird durch die folgende Gleichung beschrieben:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f). \quad (2.11)$$

Die Sigmoid-Funktion σ sorgt dafür, dass der Wert von f_t zwischen 0 und 1 liegt, sodass das Netzwerk selektiv Informationen aus dem vorherigen Zellzustand C_{t-1} behalten oder verwerfen kann.

Das Eingangsgate i_t bestimmt, welche neuen Informationen in den Zellzustand aufgenommen werden. Gleichzeitig wird ein neuer Kandidatenzustand (Candidate State)

$\tilde{C}_t$ berechnet, der als potenzielle neue Information für den Speicher dient:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i), \quad (2.12)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C). \quad (2.13)$$

Das Eingangsgate skaliert die neuen Informationen, indem es bestimmt, welche Teile des Kandidatenzustands tatsächlich in den Zellzustand übernommen werden sollen.

Die Aktualisierung des Zellzustands erfolgt durch eine Kombination der alten und neuen Informationen:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t. \quad (2.14)$$

Das Symbol $\odot$ bezeichnet die elementweise Multiplikation. Durch die gewichtete Kombination mit dem Vergessensgate kann das Modell wichtige Informationen über mehrere Zeitschritte hinweg bewahren.

Das Ausgangsgate o_t steuert, welche Informationen aus dem Zellzustand in den verborgenen Zustand h_t übertragen werden. Die Berechnung erfolgt über:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o), \quad (2.15)$$

$$h_t = o_t \odot \tanh(C_t). \quad (2.16)$$

Die nicht lineare Transformation durch die Tanh-Funktion stellt sicher, dass der Zellzustand in einen geeigneten Wertebereich skaliert wird, bevor er an die nächste Zeitschicht weitergegeben wird.

Durch diese Mechanismen sind LSTM-Netzwerke in der Lage, relevante Informationen über längere Sequenzen hinweg zu speichern und irrelevante Informationen gezielt zu vergessen. Dies macht sie besonders geeignet für Anomalieerkennung in Zeitreihen [15]. Ein schematisches Diagramm der LSTM-Architektur findet sich in der Arbeit von Le et al. (2019), in der die Funktionsweise eines LSTM-Netzwerks für Hochwasservorhersagen visualisiert wird [28].

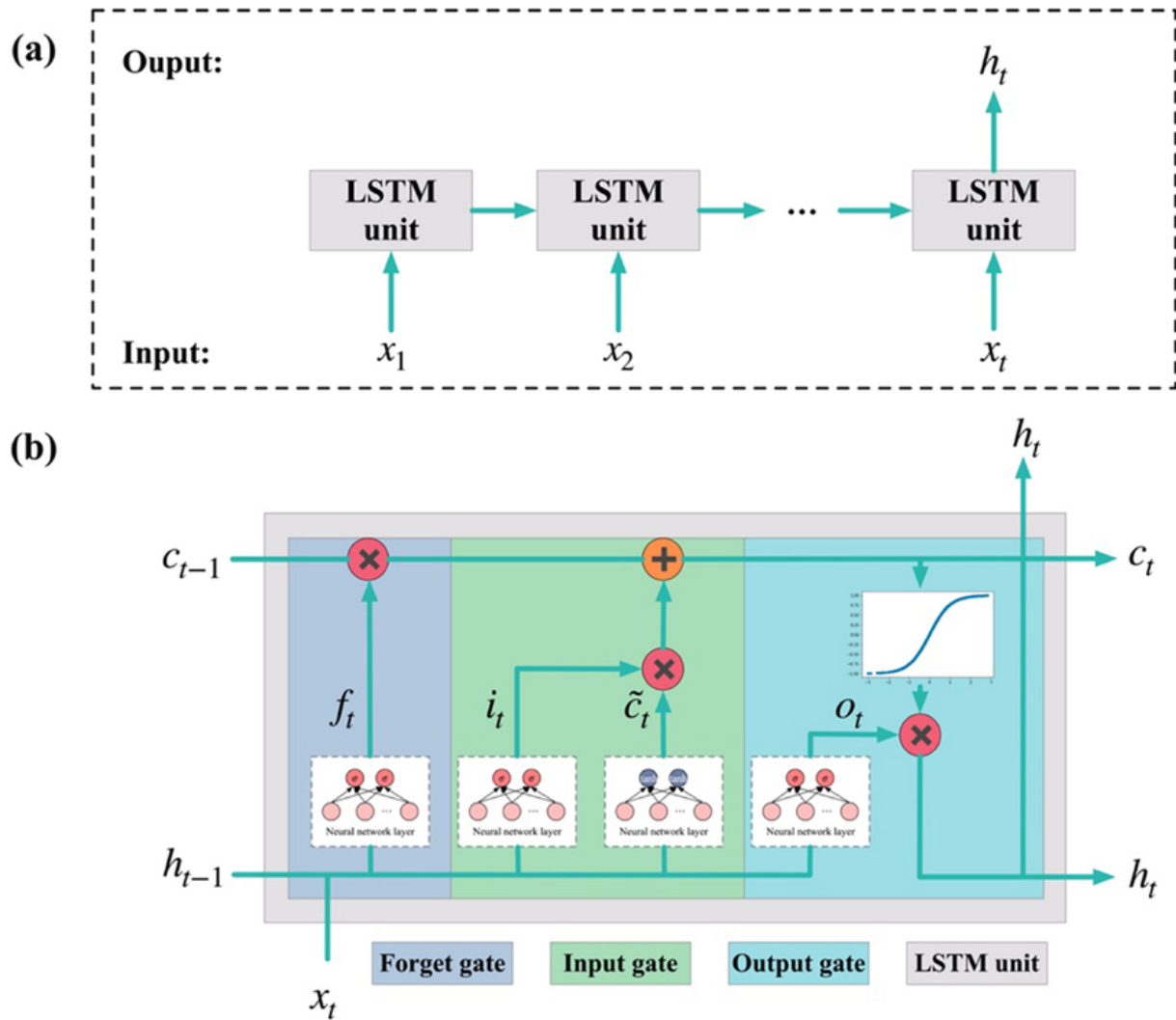


Abbildung 2.5: Architektur einer LSTM-Zelle [28]

Das Paper von Olah, Christopher "Understanding LSTMs"(2015) gibt eine detailliertere Einführung in LSTMs, da es die Funktionsweise dieser Netzwerke auf anschauliche und visuelle Weise erklärt [29].

2.6.1 Bidirektionale Netzwerke

Während herkömmliche LSTMs Informationen nur in eine Richtung propagieren, sind bidirektionale LSTMs (BiLSTMs) in der Lage, Informationen sowohl aus der Vergangenheit als auch aus der Zukunft einer Sequenz zu erfassen. Dies wird durch zwei separate LSTM-Schichten erreicht: eine vorwärtsgerichtete Schicht (Forward Layer), die die Sequenz in normaler Reihenfolge verarbeitet, und eine rückwärtsgerichtete Schicht (Backward Layer), die die Sequenz in umgekehrter Reihenfolge verarbeitet [30].

Die Architektur eines BiLSTM-Netzwerks besteht aus zwei parallel verlaufenden

LSTM-Schichten, die unabhängig voneinander trainiert werden. Für eine gegebene Eingangssequenz $X = (x_1, x_2, \dots, x_T)$ berechnet die vorwärtsgerichtete LSTM-Schicht eine versteckte Zustandsfolge $\vec{h}_t$, während die rückwärtsgerichtete LSTM-Schicht eine separate Sequenz $\overleftarrow{h}_t$ erzeugt:

$$\vec{h}_t = LSTM_{\text{forward}}(x_t, \vec{h}_{t-1}) \quad (2.17)$$

$$\overleftarrow{h}_t = LSTM_{\text{backward}}(x_t, \overleftarrow{h}_{t+1}) \quad (2.18)$$

Durch die Kombination beider Zustände entsteht der finale Repräsentationsvektor:

$$h_t = [\vec{h}_t; \overleftarrow{h}_t] \quad (2.19)$$

wobei $[\cdot; \cdot]$ die Konkatenation (Verkettung) der beiden Vektoren bezeichnet. Das bedeutet, dass der resultierende Vektor h_t einfach die beiden Teilvektoren aneinanderreih, sodass die Dimension des kombinierten Vektors doppelt so groß ist wie die ursprüngliche Dimension eines einzelnen Zustands. Diese Konstruktion erlaubt es dem Modell, Informationen aus dem gesamten Kontext einer Sequenz zu nutzen und somit eine umfassendere Repräsentation der Daten zu lernen.

BiLSTMs haben sich in der Zeitreihenanalyse als vorteilhaft erwiesen, da sie sowohl vergangene als auch zukünftige Trends berücksichtigen können [15].

In Abbildung 2.6 ist eine schematische Darstellung einer Architektur eines BiLSTMs dargestellt.

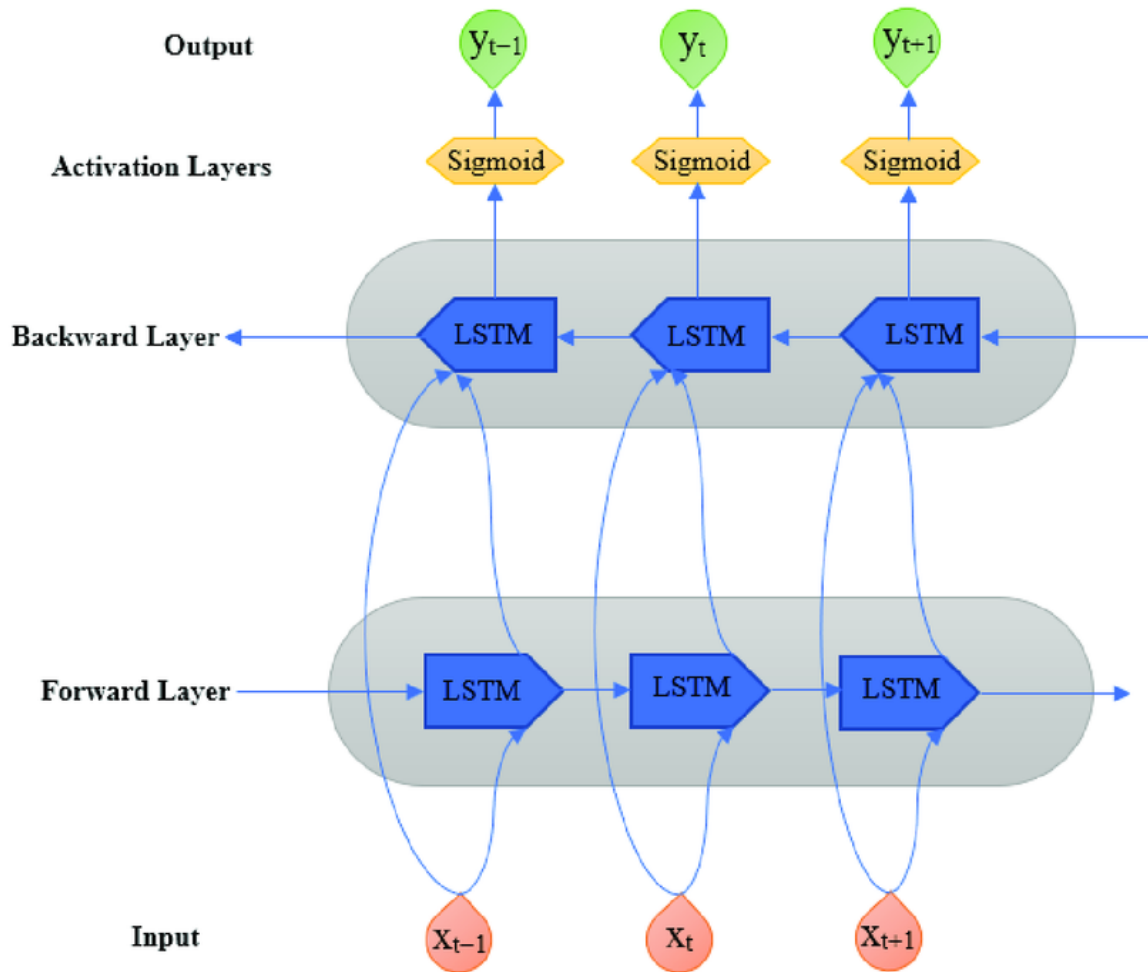


Abbildung 2.6: Architektur eines bidirektionalen LSTM-Netzwerks mit Sigmoid Aktivierungsfunktion [30]

2.7 Attention Heads

Attention-Mechanismen sind eine zentrale Technik in modernen Deep-Learning-Modellen, die es ermöglichen, wichtige Teile einer Eingabesequenz gezielt zu gewichten. Anstatt alle Eingabeelemente gleich zu behandeln, berechnet eine Attention-Schicht für jedes Token oder jeden Zeitschritt eine Gewichtung, die dessen Relevanz für die aktuelle Berechnung widerspiegelt. Dieser Mechanismus hat sich insbesondere in den Transformer-Netzwerken bewährt, in denen Multi-Head Attention eingesetzt wird, um verschiedene Aspekte einer Sequenz parallel zu modellieren [6].

Ein einzelner Attention Head basiert auf der Berechnung von drei Vektoren für jede Eingabe: Query (Q), Key (K) und Value (V). Diese entstehen durch lineare Transformationen der

Eingabesequenz:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V. \quad (2.20)$$

Die Relevanz eines Elements innerhalb der Sequenz wird durch das Skalarprodukt zwischen den Query- und Key-Vektoren bestimmt:

$$A = \frac{QK^T}{\sqrt{d_k}}. \quad (2.21)$$

Je höher dieser Wert, desto stärker ist die Beziehung zwischen zwei Sequenzpunkten. In klassischen Transformer-Modellen wird A durch eine Softmax-Funktion normalisiert, um eine Wahrscheinlichkeitsverteilung zu erhalten. Für Anwendungen wie die Anomalieerkennung ist jedoch keine explizite Normalisierung erforderlich, da hierbei keine Klassifikation erfolgt. Stattdessen kann eine direkte Gewichtung der relevanten Sequenzbereiche vorgenommen werden:

$$\alpha = \tanh(A). \quad (2.22)$$

Der finale Ausgabevektor ergibt sich aus der gewichteten Summe der Value-Vektoren:

$$\text{Attention}(X) = \alpha V. \quad (2.23)$$

Obwohl Transformer-Modelle ausschließlich auf Attention-Mechanismen basieren, zeigen zahlreiche Studien, dass die Kombination rekurrenter Architekturen wie BiLSTMs mit Attention-Schichten leistungsfähiger ist als der isolierte Einsatz einer der beiden Methoden. Während BiLSTMs sequentielle Abhängigkeiten erfassen, ermöglicht die Attention-Schicht eine gezielte Fokussierung auf relevante Sequenzteile. In der Arbeit von Mishra et al. (2021) wird ein Attention-based BiLSTM für die Anomalieerkennung untersucht. Das Modell zeigt eine verbesserte Präzision, da es die Fähigkeit von LSTMs zur Modellierung langzeitlicher Abhängigkeiten mit der flexiblen Gewichtung durch Attention kombiniert [31]. Ebenso demonstriert Lee (2022), dass die Integration von Attention in LSTMs bei Zeitreihenprognosen zu einer signifikanten Reduktion von Vorhersagefehlern führt [32].

Die Anwendung von Attention Heads in BiLSTMs ermöglicht eine adaptive Selektion relevanter Informationen innerhalb einer Sequenz. Während rekurrente Netzwerke typischerweise die gesamte Zeitschrittfolge durchlaufen, kann eine Attention-Schicht

wichtige Zeitschritte hervorheben und weniger relevante Informationen abschwächen. Dadurch wird die Erklärbarkeit des Modells verbessert und die für die Vorhersage entscheidenden Merkmale effizienter extrahiert.

Multi-Head Attention

Während eine einzelne Attention-Schicht bereits eine gezielte Gewichtung relevanter Sequenzabschnitte ermöglicht, bietet der Einsatz von Multi-Head Attention zusätzliche Vorteile. Anstatt sich nur auf eine einzige Sichtweise der Daten zu beschränken, kann das Modell mehrere verschiedene Repräsentationen der Eingabesequenz gleichzeitig lernen. Dies verbessert insbesondere die Erfassung komplexer Muster, die für die Anomalieerkennung entscheidend sind.

Ein wesentlicher Vorteil von Multi-Head Attention besteht in der besseren Steuerung des Fokus. Klassische LSTM-Modelle gewichten alle Zeitpunkte einer Sequenz ähnlich, während Attention spezifische Abschnitte hervorheben kann. Besonders bei der Anomalieerkennung ist dies relevant, da anomale Ereignisse oft nur in kurzen Zeitfenstern auftreten und eine gleichmäßige Verarbeitung der gesamten Sequenz dazu führen kann, dass diese wichtigen Abweichungen nicht ausreichend berücksichtigt werden.

Multi-Head Attention erweitert den klassischen Attention-Mechanismus, indem mehrere unabhängige Attention-Schichten parallel berechnet werden. Die einzelnen Attention-Ausgaben werden anschließend zu einer Gesamtrepräsentation zusammengeführt:

$$\text{MultiHead}(X) = \text{Concat}(\text{Attention}_1(X), \dots, \text{Attention}_h(X))W_O. \quad (2.24)$$

Hierbei bezeichnet h die Anzahl der Attention-Heads, $\text{Attention}_i(X)$ die Ausgabe des i -ten Attention-Heads und W_O eine Gewichtungsmatrix zur Transformation der kombinierten Repräsentation [6].

Diese parallele Verarbeitung ermöglicht eine Erfassung unterschiedlicher Abhängigkeiten innerhalb der Sequenz, da jeder Attention-Head auf verschiedene Aspekte der Eingabedaten fokussieren kann. Dadurch wird nicht nur die Modellkapazität erhöht, sondern auch die Robustheit gegenüber Variationen in den Daten verbessert [33].

Schließlich profitiert das Modell von der Fähigkeit, mehrere Perspektiven gleichzeitig zu analysieren. Während eine einzelne Attention-Schicht nur eine bestimmte Art

von Beziehung zwischen Sequenzelementen erfasst, können mehrere Attention-Heads unterschiedliche Merkmale gleichzeitig lernen. Studien zeigen, dass Multi-Head Attention insbesondere in Kombination mit rekurrenten Architekturen, wie BiLSTMs, signifikante Verbesserungen bei Zeitreihenanalysen und Anomalieerkennungsaufgaben bietet [31], [32].

2.8 Modellfabrik

Für die Daten des realen Datensatzes wurde ein Teilprozess, der im Otto Rettenmaier Forschungslabor stehenden Modellfabrik, verwendet. Diese Anlage wird im Folgenden kurz beschrieben.

Die Anlage wurde von der Firma Festo Didactic bereitgestellt. Hierbei wird ein Abfüllprozess von Flüssigkeiten und Feststoffen in kleine Flaschen sowie der Transport und das Recycling der Flaschen dargestellt, sodass ein Kreislauf entsteht [7]. In Abbildung 2.7 ist diese dargestellt.

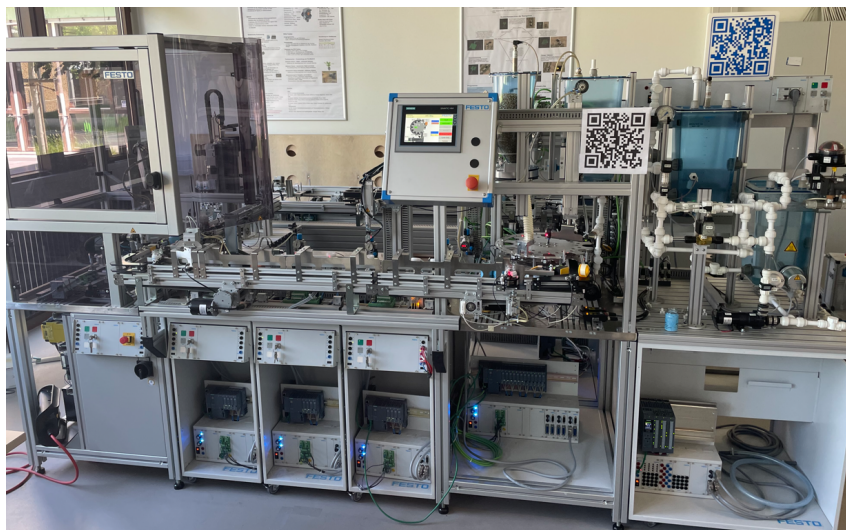


Abbildung 2.7: Modellfabrik der HHN im Otto Rettenmaier Forschungslabor [7]

Für die Anomalieerkennung wurde nur der Prozess der Filling-Station verwendet, da dieser dem verwendeten theoretischen Datensatz am ähnlichsten ist (siehe Kapitel 4.3).

Die Filling-Station setzt sich aus einem Transportband (rote zwei) und einem Drehtisch (orangene eins) zusammen, wie in Abbildung 2.8 dargestellt. Der Drehtisch führt die einzelnen Flaschen durch die verschiedenen Teilschritte des Abfüllprozesses, darunter das Füllen und Verschließen. Allerdings kann nicht jeder dieser Schritte lückenlos überwacht werden, da für einige Prozessschritte keine analogen Sensordaten verfügbar sind.

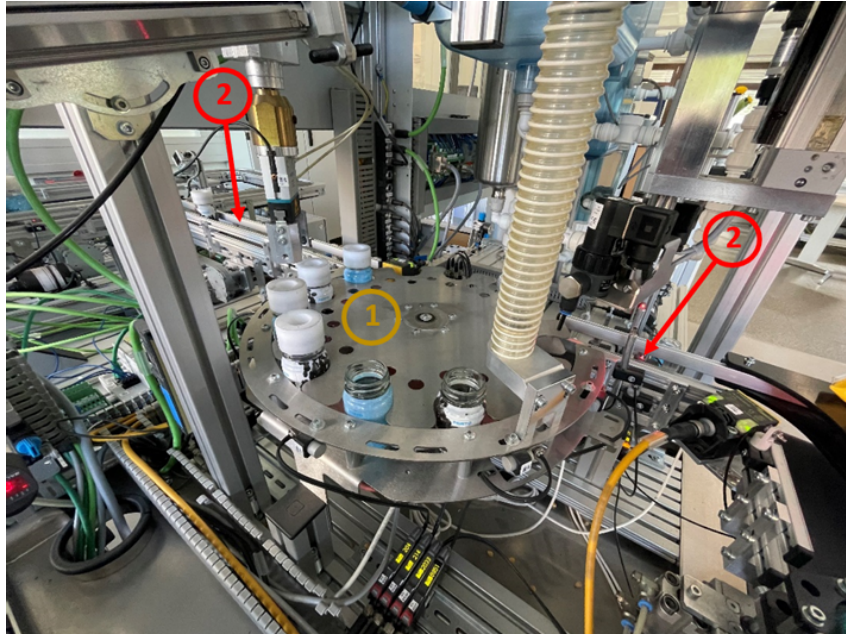


Abbildung 2.8: Filling-Station der Modellfabrik [7]

Für die Anomalieerkennung stehen die in 2.1 aufgeführten Variablen zur Verfügung. Besonders relevant sind dabei die folgenden Analogwerte: der Motorstrom der Förderbänder, der Motorstrom des Drehtisches sowie der Gesamtstromverbrauch der Anlage.

Name	Typ (DT)	Name	Typ (DT)
Timestamp	Int32	Position des Separators für Flüssigkeiten geschlossen	Bool
Drucksensor Pneumatik	Int32	Position des Separators für Feststoffe geschlossen	Bool
Gesamtstrom	Int32	Ansteuerung des Separators für Flüssigkeiten	Bool
Motorstrom am Transportförderband	Int32	Ansteuerung des Separators für Feststoffe	Bool
Motorstrom am Drehtisch	Int32	Ansteuerung des Flaschenfixierers	Bool
Motorstrom am Zufuhrförderband	Int32	Ansteuerung der Greiferbacken	Bool
Raumtemperatur	Int32	Einleitung der Greiferverfahrensbewegung nach unten	Bool
Temperatur am Motorgehäuse des Transportförderbands	Int32	Einleitung der Greiferdrehung auf 0°	Bool
Ansteuerung Förderband	Bool	Einleitung der Greiferdrehung auf 270°	Bool
Ansteuerung Drehtisch	Bool		
Ansteuerung Zufuhrförderband	Bool		
Obere Position Greifer erreicht	Bool		
Untere Position Greifer erreicht	Bool		

Tabelle 2.1: Sensor- und Steuervariablen der Filling-Station [7]

3 Verwandte Arbeiten

3.1 BiLSTM Autoencoder

Ein BiLSTM Autoencoder nutzt bidirektionale LSTM-Schichten im Encoder und Decoder, um Sequenzdaten aus beiden Zeitrichtungen zu verarbeiten [34]. Raihan und Ahmed zeigen, dass die Bidirektionalität hilft, lange zeitliche Abhängigkeiten und Muster in Zeitreihen effektiver zu erfassen als unidirektionale LSTM-Modelle. Der Autoencoder wird auf normalem Verhalten trainiert und versucht, Eingabesequenzen möglichst originalgetreu zu rekonstruieren. Anomalien lassen sich anschließend durch einen erhöhten Rekonstruktionsfehler erkennen, da abweichende Muster vom Modell schlechter reproduziert werden. In einer Fallstudie auf einem Windpark-Datensatz erreichte der BiLSTM-AE eine Klassifikationsgenauigkeit von rund 96,79% und übertraf damit herkömmliche LSTM-Autoencoder deutlich, was den zusätzlichen Nutzen der bidirektionalen Verarbeitung unterstreicht [34].

3.2 BiLSTM Variational Autoencoder

Ein BiLSTM-VAE kombiniert einen Variational Autoencoder – bei dem der Encoder statt eines deterministischen Codes eine Wahrscheinlichkeitsverteilung ausgibt – mit BiLSTM-Schichten [35]. Dieser probabilistische Ansatz bildet Unsicherheit in der Rekonstruktion explizit ab: Normalzustände werden in einen kompakten, regelmäßigen Latenten Raum eingebettet, während Anomalien durch höhere Rekonstruktionsfehler und abweichende latente Verteilungen auffallen. Die BiLSTM-Komponenten erfassen zusätzlich kontextuelle Informationen aus beiden zeitlichen Richtungen. In Experimenten mit Herzfrequenz-Zeitserien berichten Staffini et al., dass ihr BiLSTM-VAE anderen Methoden in Bezug auf Genauigkeit und Robustheit überlegen war. Besonders hervorzuheben ist das Gleichgewicht zwischen hoher Präzision und solidem Recall, das in den Testszenarien erreicht wurde [35].

3.3 Attention-based BiLSTM Autoencoder

Ein Attention-based BiLSTM-Autoencoder erweitert das Encoder-Decoder-Konstrukt um einen Attention-Layer. Kim et al. [36] integrieren hierfür ein Attention-Modul, das für jeden Zeitschritt Gewichtungsfaktoren lernt und damit die relevanten Zeitpunkte im Rekonstruktionsprozess stärker hervorhebt. Dies ermöglicht eine gezielte Fokussierung auf auffällige Muster oder plötzliche Ausreißer, anstatt alle Zeitschritte gleich zu behandeln. Die lokalen Informationen jeder Zeitinstanz bleiben durch die bidirektionalen LSTM-Schichten erhalten und werden gemäß ihrer Wichtigkeit skaliert. In empirischen Tests, beispielsweise mit Daten aus einem industriellen Prozess, erzielte das Attention-based BiLSTM-AE eine Fehlerkennungsrate von über 95%, was die Wirksamkeit des Aufmerksamkeitsmechanismus bei der Unterscheidung zwischen Normal- und Anomalieverläufen belegt [36].

4 Daten

In folgendem Kapitel werden die verwendeten Datensätze genauer erläutert. Bevor die Datensätze jedoch vorgestellt werden, wird die Datenaufnahme an der Modellfabrik kurz erläutert. Zusätzlich wird die Datenvorverarbeitung mit der verwendeten Preprocessing-Library (TimeSeriesPreprocessingPipeline) [7] genauer erläutert. Anschließend wird der theoretische Datensatz TEP und der reale Datensatz der Modellfabrik erläutert.

4.1 Datenaufnahme an der Modellfabrik

Die Datenaufnahme des realen Datensatzes der Filling-Station wurde nicht im Rahmen dieser Arbeit durchgeführt, sondern von Schoch im Zuge seiner Masterarbeit [7]. Dabei wurde ein bestehendes Python-Programm genutzt, um die Daten über OPC-UA aus der SPS-Steuerung auszulesen und in CSV-Dateien zu speichern. Um eine spätere Echtzeitübertragung zu ermöglichen, wurde eine Connector-Klasse hinzugefügt, die die Daten über TCP an andere Teilnehmer sendet. Da OPC-UA nicht echtzeitfähig ist, wurde ein Ringpuffer mit Zeitstempel in der SPS verwendet, der regelmäßig ausgelesen wird, um Datenverluste zu vermeiden. Eine detaillierte Beschreibung der Datenaufnahme kann in der Masterarbeit von Schoch nachgelesen werden [7].

4.2 Datenvorverarbeitung

Für die Datenvorverarbeitung wurde die zur Verfügung gestellte Preprocessing-Library von Schoch verwendet. Diese beinhaltet fast alle benötigten Operationen, um die Daten vorzuverarbeiten. Hierbei können folgende Operationen durchgeführt werden:

- Erhöhung der Abtastrate (Downsampling)
- Skalierung zwischen 0 und 1
- Aufteilung des Datensatzes in überlappende Fenster (Sliding Window)
- Auswahl der Variablen, die als Modelleingabe dienen (X-Daten)
- Auswahl der Variablen, die als Modellausgang / Ground-Truth dienen (Y-Daten)
- Maskieren der Modelleingabe und Vervielfältigung der Daten

Im Folgenden wird auf die einzelnen Punkte genauer eingegangen.

4.2.1 Downsampling

Downsampling ist eine Technik zur Reduktion der Abtastrate, um die Datenmenge zu verringern und den Rechenaufwand zu reduzieren, ohne wesentliche Merkmale der Daten zu verlieren. Besonders in industriellen Anwendungen kann es vorkommen, dass die ursprüngliche Abtastrate höher ist, als für die Analyse erforderlich. Durch die Anpassung der Abtastrate kann eine effizientere Verarbeitung der Zeitreihen ermöglicht werden, indem unnötige Datenpunkte entfernt werden, während die relevanten Informationen erhalten bleiben.

Die Implementierung der Funktion für Analogwerte erfolgte mit Hilfe der Funktion `resample` von Scipy. Diese Funktion basiert auf der Fourier-Transformation, sodass unter anderem hochfrequente Anteile herausgefiltert werden. Boolesche Werte würden durch die Anwendung jedoch stark verändert, daher wird bei booleschen Werten die Änderung der Abtastrate durch das Auslassen der entsprechenden Datenpunkte realisiert.

4.2.2 Skalierung

Deep-Learning-Netzwerke arbeiten am effizientesten mit Zahlen im Bereich von 0 bis 1, da dies das Gradient-Vanishing-Problem reduziert [26]. Daher werden die Daten vor der Modellverarbeitung auf diesen Bereich skaliert. Für jede Variable werden die minimalen und maximalen Werte der Trainingsdaten gespeichert, um eine konsistente Skalierung sicherzustellen. Die Testdaten werden anschließend anhand dieser Werte transformiert.

Es kann jedoch vorkommen, dass der Wertebereich der Testdaten nach der Skalierung außerhalb des Bereichs von 0 bis 1 liegt, insbesondere wenn die Testdaten Anomalien enthalten. In diesem Fall können Werte außerhalb der zuvor definierten Grenzen auftreten. Wichtig zu erwähnen ist, dass die eingesetzte Preprocessing-Library ausschließlich den `MinMaxScaler` unterstützt, wodurch alternative Skalierungsmethoden, wie den `StandardScaler`, nicht genutzt werden können und außerhalb der Library definiert werden müssen.

4.2.3 Sliding Windows

Zeitreihen enthalten oft lange, kontinuierliche Datenfolgen, die nicht direkt von rekonstruktionsbasierten Modellen verarbeitet werden können. Daher werden sie in kleinere, überlappende Segmente unterteilt, sogenannte Sliding Windows. Dies ermöglicht es dem Modell, zeitliche Abhängigkeiten besser zu lernen und eine präzisere Anomalieerkennung durchzuführen [26].

Ein Sliding Window wird durch zwei Parameter definiert: die Fenstergröße und den Stride. Während die Fenstergröße bestimmt, wie viele Zeitschritte pro Segment betrachtet werden, legt der Stride fest, um wie viele Zeitschritte das nächste Fenster versetzt ist. Ein größerer Stride reduziert die Anzahl der verarbeiteten Fenster, während ein minimaler Stride eine maximale Überlappung der Segmente bewirkt. Eine schematische Darstellung dieses Prinzips ist in Abb. 4.1 zu sehen.

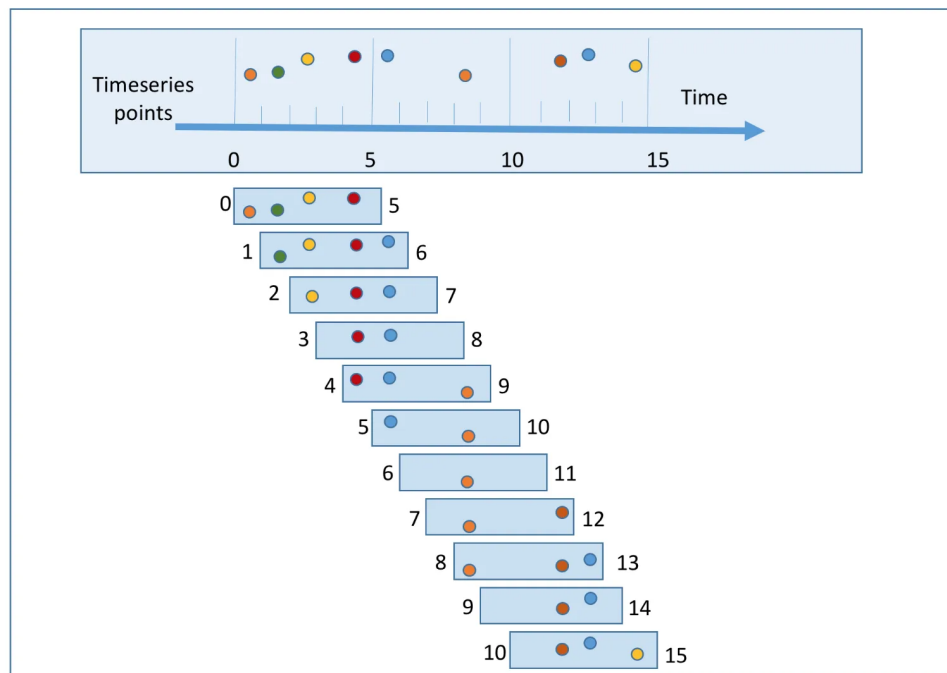


Abbildung 4.1: Illustration des Sliding Window-Verfahrens [37]

Für die Implementierung wurde NumPy verwendet, um eine effiziente Verarbeitung der Segmente zu gewährleisten. Die Wahl der Fenstergröße und des Strides wurde empirisch ermittelt, um die Modellleistung zu maximieren [37], [38].

4.2.4 Auswahl der Variablen

Nicht alle verfügbaren Variablen enthalten relevante Informationen für die Anomalieerkennung. Daher gibt es eine Auswahlmethode, mit der festgelegt werden kann, welche Variablen in das Modell eingehen und welche rekonstruiert werden sollen. Auf diese Weise können irrelevante oder wenig aussagekräftige Variablen ausgeschlossen werden, was die Modellleistung und Interpretierbarkeit verbessert.

Die gezielte Auswahl relevanter Variablen reduziert die Modellkomplexität, verringert den Rechenaufwand und erhöht die Aussagekraft der Ergebnisse. Zudem wird das Risiko minimiert, dass unnötige oder verrauschte Daten die Anomalieerkennung negativ beeinflussen.

Die Implementierung dieser Funktion wurde mit NumPy realisiert, um eine effiziente Filterung der Eingangsdaten zu gewährleisten.

4.2.5 Maskierung

Die Maskierung von Teilen der Eingabedaten hat sich als eine effektive Methode erwiesen, um Deep-Learning-Modelle für die Rekonstruktion zu trainieren. Ursprünglich wurde dieses Verfahren in der natürlichen Sprachverarbeitung (Natural Language Processing, NLP) eingesetzt, insbesondere für das Training großer Sprachmodelle wie Bidirectional Encoder Representations from Transformers (BERT) [39]. Später fand die Technik auch Anwendung in der Anomalieerkennung von Zeitreihendaten [40].

In der vorliegenden Arbeit wurde die Maskierung mit NumPy implementiert, wobei in Anlehnung an BERT 15% der Eingabewerte nicht maskiert, 15% mit zufälligen Werten ersetzt und die verbleibenden 70% auf 0 gesetzt werden. Die Länge der Maskierung wird als Parameter definiert, während die Positionen zufällig bestimmt werden. Durch die Maskierung kann zudem eine Vervielfältigung der Daten erfolgen, um verschiedene Maskierungen für denselben Datenpunkt zu generieren. Dies erhöht die Datenmenge für das Training und verbessert die Generalisierungsfähigkeit des Modells.

Eine bessere Beschreibung dieses Ansatzes und dessen Anwendung auf industrielle Zeitreihendaten kann in der Masterarbeit von Schoch nachgelesen werden [7].

4.3 TEP-Datensatz

Zur Validierung des verwendeten Modells und zum Vergleich mit anderen Ansätzen wird in dieser Arbeit zusätzlich zu den selbst erstellten Datensätzen der öffentlich zugängliche Tennessee Eastman Process (TEP) Datensatz verwendet [38]. Der TEP-Datensatz ist ein synthetisch generierter Datensatz eines chemischen Prozesses, der häufig als Benchmark für Algorithmen zur Anomalieerkennung in Zeitreihen genutzt wird [41]. Der Datensatz besteht aus zwei Trainingsdateien und zwei Testdateien, wobei jeweils eine Datei normale Betriebsdaten und eine Datei anomale Betriebsdaten enthält. Die anomalen Daten beinhalten 20 unterschiedliche Fehlerklassen. Die Daten sind in Durchläufe unterteilt, wobei jeweils 500 Durchläufe für den Normalzustand und für jede der 20 Anomalien mit zufälligen Startwerten durchgeführt wurden. Ein Durchlauf der Trainingsdaten umfasst 500 Zeitschritte mit einer Abtastrate von drei Minuten, was einer Gesamtdauer von 25 Stunden entspricht. Die Testdaten bestehen hingegen aus 960 Zeitschritten, was einer Gesamtdauer von 48 Stunden entspricht [42].

Der Datensatz enthält insgesamt 55 Variablen. Die erste Variable gibt die Nummer des Durchlaufs an, die zweite die Anomalieklasse und die dritte den Zeitschritt. Die verbleibenden 52 Variablen stellen verschiedene Messwerte dar. Zur Sicherstellung der Modellgeneralisation werden 30% der Trainingsdaten für die Validierung verwendet. Ein wichtiger Aspekt bei der Datenaufbereitung ist die Skalierung der Werte, da die Daten in sehr unterschiedlichen Größenordnungen vorliegen. Die vorliegende Preprocessing-Library [7] unterstützt lediglich eine Skalierung mit dem MinMax-Scaler. Es hat sich durch empirische Herangehensweise herausgestellt, dass die Skalierung mit dem StandardScaler ein wesentlich besseres Ergebnis liefert. Aus diesem Grund wurden die Daten außerhalb der Preprocessing-Library skaliert.

Der TEP-Datensatz wurde aufgrund seiner strukturierten Anomalieklassen, seiner industriellen Relevanz und seiner Verwendung als Benchmark-Datensatz für Anomalieerkennungsalgorithmen ausgewählt. Zudem ermöglicht er einen Vergleich mit bestehenden nicht-transformerbasierten Anomalieerkennungsmodellen [43]. Dies bietet die Möglichkeit, die Leistungsfähigkeit der hier entwickelten BiLSTM-Autoencoder-, BiLSTM-VAE- und Attention-based Modelle zu validieren und mit etablierten Methoden zu vergleichen.

4.3.1 Korrelierende Daten

Eine Analyse der Messwerte zeigt, dass einige Variablen verrauscht sind oder diskrete Werte annehmen. Beispielsweise wechselt die Variable `xmeas_9` konstant zwischen festen Werten, was darauf hinweist, dass es sich möglicherweise um einen binären oder kategorischen Sensor handelt.

Eine explorative Datenanalyse zeigt, dass einige Variablen stark korreliert sind. Insbesondere bestehen hohe Korrelationen zwischen bestimmten gemessenen Variablen (`xmeas_5`) und kontrollierten Variablen (`xmv`), was darauf schließen lässt, dass bestimmte Steuergrößen direkten Einfluss auf die Prozesswerte haben. Diese Korrelationen werden in der nachfolgenden Korrelationsmatrix visualisiert (siehe Abbildung 4.2).

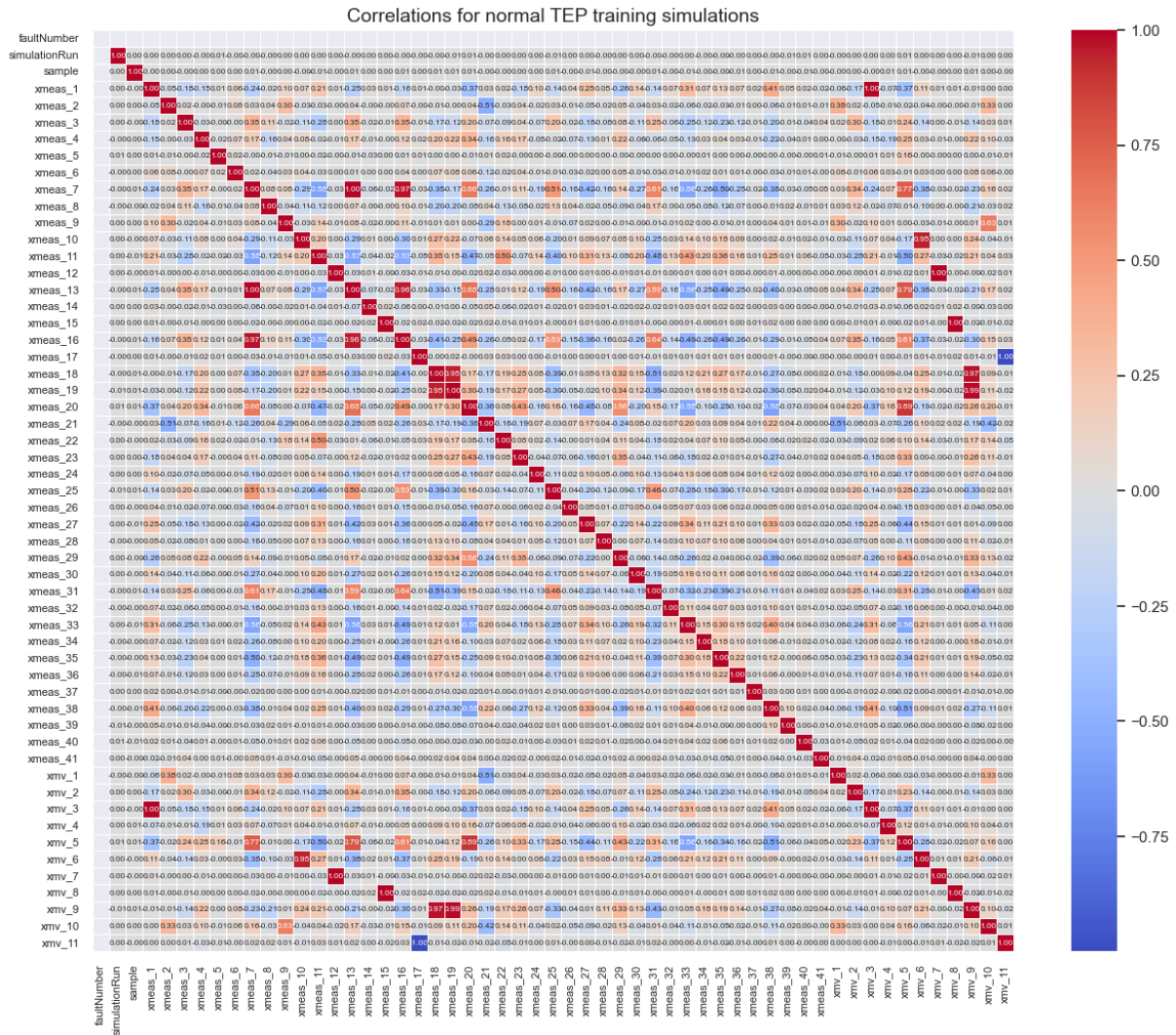


Abbildung 4.2: Korrelationsmatrix der relevantesten Variablen im TEP-Datensatz. Starke Korrelationen sind in dunklen Farben markiert.

Besonders auffällig sind die folgenden Paare:

Variable 1		Variable 2
xmeas_7	$\Leftrightarrow$	xmeas_13
xmeas_16	$\Leftrightarrow$	xmeas_13
xmeas_19	$\Leftrightarrow$	xmv_9
xmeas_10	$\Leftrightarrow$	xmv_6
xmeas_20	$\Leftrightarrow$	xmv_5
xmeas_12	$\Leftrightarrow$	xmv_7
xmeas_1	$\Leftrightarrow$	xmv_3
xmeas_17	$\Leftrightarrow$	xmv_11

Tabelle 4.1: Korrelationen zwischen Variablen im TEP-Datensatz

Aufgrund der Korrelation der Variablen, werden diese aus dem Datensatz entfernt, sodass die Trainingsleistung des Modells nicht beeinflusst wird.

4.3.2 Daten Visualisierung

Die Anomaliendaten und die Normaldaten weisen zu Beginn eine identische Struktur auf, wie in Abbildung 4.3 dargestellt. Dieses Phänomen tritt bei vielen Anomalien auf und führt dazu, dass in diesen Bereichen keine Abweichungen festgestellt werden können. Dadurch wird eine vollständige Erkennung von Anomalien mittels eines punktbasierten Anomalie-Scores unmöglich. Um dieses Problem zu umgehen, wird neben der punktbasierten Anomalieerkennung zusätzlich eine fensterbasierte Methode eingesetzt, die eine umfassendere Analyse ermöglicht. Eine detaillierte Beschreibung dieser Vorgehensweise findet sich in Abschnitt 5.1.

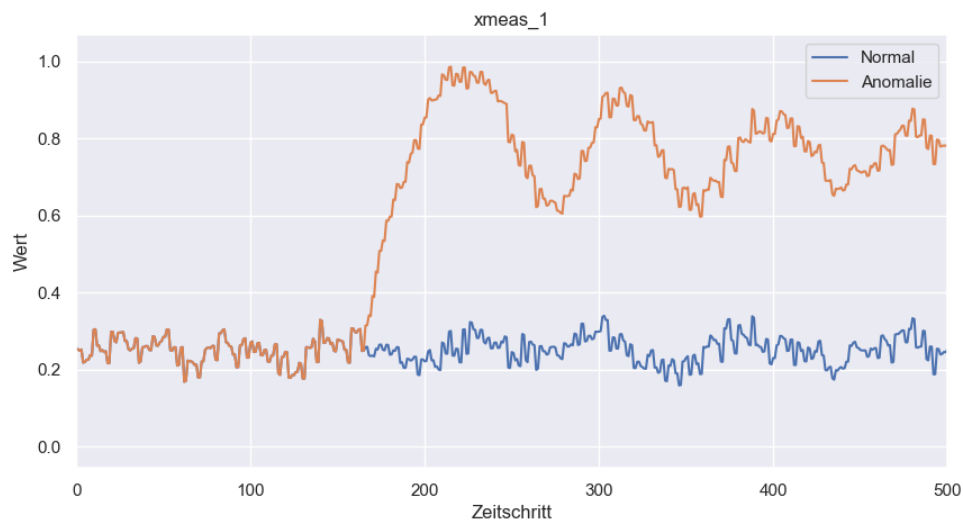


Abbildung 4.3: Normale und Anomale Daten der Variable xmeas_1

4.3.3 Vorverarbeitungsparameter

In Tabelle 4.2 sind die Parameter für die Vorverarbeitung der Daten aufgelistet.

Operation	Parameter
Sliding Window size	400
Sliding Window stride	100
Skalierung	False
Maskierung	False
Verwendete Variablen als Eingang	Variable 3-55
Verwendete Variablen zur Rekonstruktion	Variable 4-55

Tabelle 4.2: Parameter der Datenvorverarbeitung des TEP-Datensätzen

Die Größe des Sliding Windows wurde auf 400 festgelegt, da größere Fenster durch empirische Ermittlung zu besseren Ergebnissen geführt haben. Da jeder Durchlauf der Testdaten nur 500 Zeitschritte umfasst, wäre dies die theoretische Obergrenze für die Fenstergröße. Allerdings wurde bewusst eine geringere Fenstergröße gewählt, um eine flexiblere Handhabung der Testdurchläufe zu ermöglichen, da diese mit 960 Zeitschritten länger sind als die Trainingsdurchläufe. Zudem würde eine Fenstergröße von 500 nur ein einziges Fenster pro Durchlauf erzeugen. Mit einer gewählten Fenstergröße von 400 und einem Sliding Window Stride von 100 werden pro Durchlauf zehn überlappende Fenster erzeugt, was die Anzahl der Trainingsinstanzen erhöht und die Modellleistung verbessert.

Die Auswahl der Eingabe- und Rekonstruktionsvariablen erfolgte so, dass ausschließlich 52 Messwerte rekonstruiert werden, da die ersten Variablen nicht von Bedeutung sind.

Die Skalierung der Daten wurde nicht mit der Preprocessing-Library durchgeführt, da sich gezeigt hat, dass die Verwendung des Standardscalers zu besseren Ergebnissen führt. Daher wurde die Skalierung außerhalb der Library mit dem Standardscaler ausgeführt.

4.4 Filling-Station-Datensatz

Um einen praxisnahen Datensatz für die Anomalieerkennung in industriellen Anlagen zu erhalten, wurde dieser basierend auf der Filling-Station der Modellfabrik (Abschnitt 2.8) erstellt. Die Datenerfassung für diesen Anlagenteil erfolgte entsprechend der in Abschnitt 4.1 beschriebenen Methode. Insgesamt standen 22 Variablen zur Verfügung, die in Tabelle 2.1 aufgeführt sind. Die Steuerung der Anlage erfolgt mit einer SPS, die eine Zykluszeit von

6 ms besitzt, wodurch sich eine entsprechend hohe Abtastrate bei der Datenaufzeichnung ergibt.

Zur Datengenerierung wurden mehrere Aufzeichnungen des Normalzustandes an unterschiedlichen Tagen durchgeführt. Ergänzend dazu wurden gezielte Aufzeichnungen für das Anlaufen und Herunterfahren der Anlage erfasst. Zusätzlich wurde für jede Anomalie-Klasse eine separate Aufzeichnung erstellt. Der vollständige Datensatz umfasst sieben Dateien für den Normalzustand, wobei eine Datei für die Validierung (ca. 18,6 %) und eine weitere für den Test (ca. 12,3 %) genutzt werden. Die übrigen fünf Dateien wurden für das Training verwendet. Insgesamt enthält der Datensatz 584.741 Zeitschritte für den Normalbetrieb sowie 109.555 Zeitschritte für anomale Betriebszustände.

Um die Daten effizienter verarbeiten zu können, wurde die Abtastrate um den Faktor 64 reduziert. Dadurch ergeben sich für den Normalzustand etwa 9.137 Zeitschritte, was einer Zeitspanne von 3.508 Sekunden entspricht. Die anomalen Daten enthalten nach der Reduktion 1.712 Zeitschritte, entsprechend einer Gesamtdauer von 657 Sekunden [7].

4.4.1 Daten Visualisierung

Für die Erkennung von Anomalien in der Filling-Station werden hauptsächlich die analogen Variablen Gesamtstrom, Motorstrom Drehtisch und Motorstrom Förderband betrachtet, da diese die größten Auffälligkeiten bei Anomalien aufweisen. Die übrigen booleschen Steuerungs- und Sensorwerte dienen lediglich als Eingang für das Modell.

Zur Veranschaulichung sind nachfolgend einige Zeitreihen dargestellt. In Abbildung 4.4 ist der Normalzustand des Motorstroms am Drehtisch zu sehen. Die periodischen Ausschläge repräsentieren das Weiterdrehen des Drehtisches, welches notwendig ist, um die Flaschen zur nächsten Station zu transportieren.

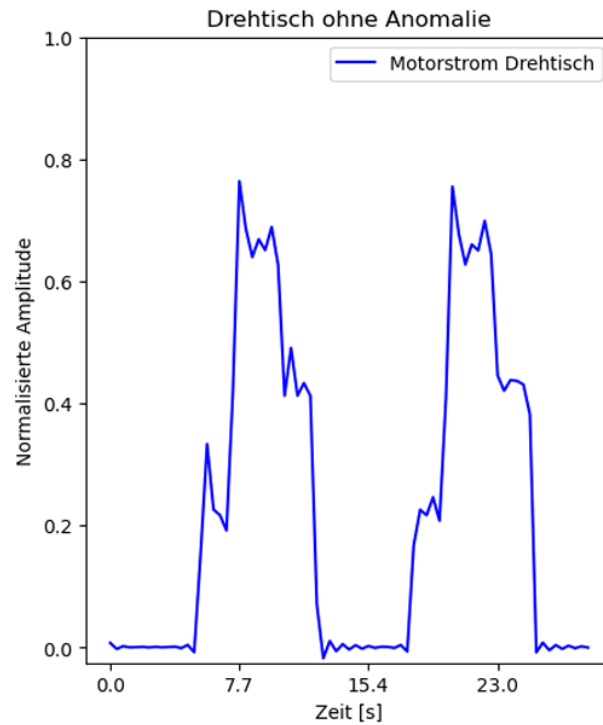


Abbildung 4.4: Motorstrom am Drehtisch im Normalzustand [7]

Abbildungen 4.5 und 4.6 zeigen zwei verschiedene Anomalien. Im ersten Fall treten anomale Drehvorgänge mit Widerstand auf, was sich in einer deutlichen Erhöhung der Stromaufnahme zeigt. Im zweiten Fall ist ein Drehvorgang mit zu geringem Widerstand dargestellt, wobei die reduzierte Stromaufnahme sichtbar wird.

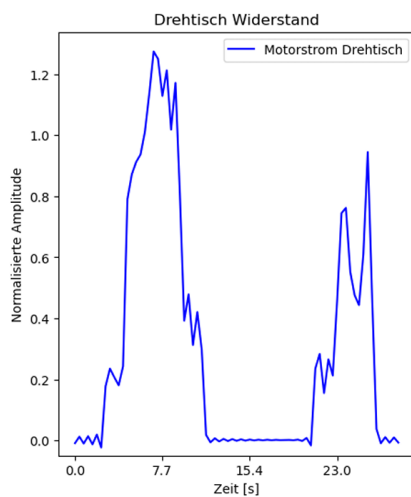


Abbildung 4.5: Drehtisch mit Widerstand [7]

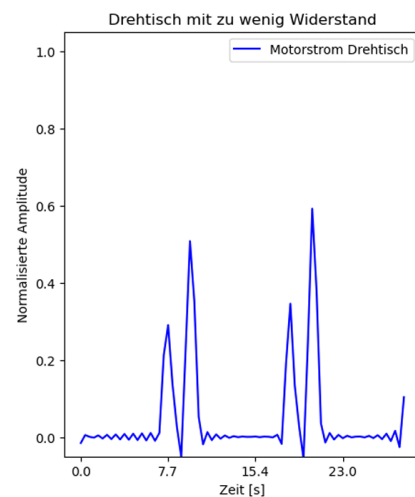


Abbildung 4.6: Drehvorgang mit zu geringem Widerstand [7]

Abbildung 4.7 zeigt den Normalzustand des Förderbands. Hier wird das Förderband je nach Zustand des Drehtisches und der Position der Flaschen zyklisch ein- und ausgeschaltet,

was sich in regelmäßigen Ausschlägen in der Stromkurve widerspiegelt. Im Gegensatz dazu zeigt Abbildung 4.8 den Stromverlauf eines Motors mit Widerstand, bei dem zusätzliche Ausschläge durch die mechanische Belastung auftreten.

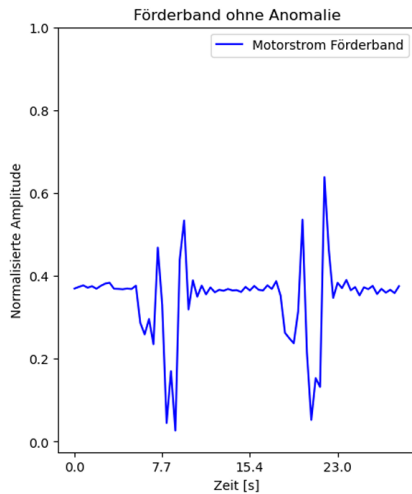


Abbildung 4.7: Motorstrom am Förderband im Normalzustand [7]

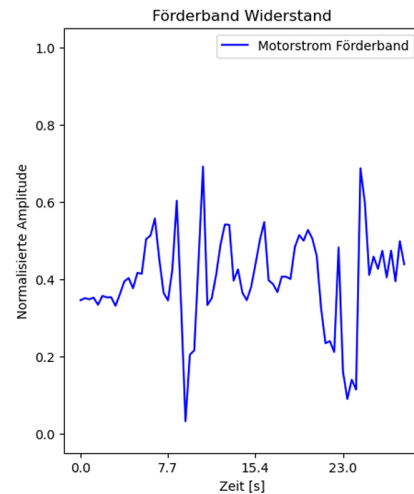


Abbildung 4.8: Motorstrom am Förderband mit Widerstand [7]

4.4.2 Vorverarbeitungsparameter

In Tabelle 4.3 sind die Parameter für die Vorverarbeitung der Daten aufgelistet

Operation	Parameter
Sliding Window size	64
Sliding Window stride	1
Downsampling Faktor	64
Skalierung	True
Maskierungslänge	0.5
Vervielfältigung mit verschiedenen Masken	4
Verwendete Variablen als Eingang	Alle
Verwendete Variablen zur Rekonstruktion	Gesamstrom Motorstrom am Förderband Motorstrom am Drehtisch

Tabelle 4.3: Parameter der Datenverarbeitung des Filling-Station-Datensatzes

Die Größe des Sliding Windows wird so gewählt, dass mindestens ein vollständiger Prozesszyklus erfasst wird. Verschiedene Werte wurden getestet, wobei die Konfiguration mit den besten Ergebnissen übernommen wurde. Um die Datenmenge und das Rauschen zu reduzieren, wird ein Downsampling-Faktor von 64 verwendet, wodurch sich eine neue

Abtastrate von 384 ms ($6 \text{ ms} * 64$) ergibt. Diese Maßnahme hat keinen negativen Einfluss auf die Anomalieerkennung, da die relevanten Prozesse im System vergleichsweise langsam ablaufen. In den nachfolgenden Diagrammen ist zu erkennen, dass die dargestellten Zeitreihen eine Fenstergröße von 64 Zeitschritten bzw. 25,6 Sekunden umfassen, wodurch mindestens ein vollständiges Fenster abgebildet wird.

Die Anwendung einer Maskierungsrate von 0.5 führte zu einer besseren Erkennungsrate, da das Modell durch diesen Ansatz den Normalzustand effektiver erlernen kann. Für die Rekonstruktion werden ausschließlich analoge Variablen herangezogen, in denen sich Anomalien manifestieren. Alle verfügbaren Variablen dienen als Eingangsdaten für das Modell, wodurch die Anzahl der Parameter nur minimal erhöht wird. Dies ermöglicht es dem Modell, die für die Anomalieerkennung relevantesten Variablen eigenständig zu identifizieren. Weitere Details zur Vorverarbeitung sind in Abschnitt 4.1 beschrieben [7].

5 Methodik

In diesem Kapitel wird die Methodik der vorliegenden Arbeit dargelegt. Zunächst erfolgt die Erläuterung des Verfahrens zur Berechnung des Anomalie-Scores. Im Anschluss werden die verwendeten Modelle vorgestellt und ihre jeweiligen Architekturen beschrieben. Abschließend erfolgt eine Erklärung der Trainingsstrategie sowie der Evaluierungsmethoden, um die Leistungsfähigkeit der Modelle systematisch zu analysieren.

5.1 Anomalieerkennung

In dieser Arbeit wird die Rekonstruktionsmethode zur Erkennung von Anomalien eingesetzt. Diese Methode wurde gewählt, da sie sich in der Literatur als effektives Verfahren zur Anomalieerkennung erwiesen hat. Für die Rekonstruktion werden neuronale Netze verwendet, die in der Lage sind, die Eingabedaten oder Teile davon wiederherzustellen. Die in dieser Arbeit verwendeten Modelle sind in Abschnitt 5.2 detailliert beschrieben [44].

Die Anomalieerkennung erfolgt in einem semi-supervised Ansatz, bei dem die Modelle ausschließlich mit nicht-anomalen Daten trainiert werden. Ziel ist es, den Normalzustand so genau wie möglich zu rekonstruieren, während abweichende, anomale Muster schlechter rekonstruiert werden. Der Trainingsprozess wird durch den MSE als Verlustfunktion gesteuert, der gleichzeitig als Maß für die Qualität der Rekonstruktion dient [14].

Zur Detektion von Anomalien wird die Rekonstruktion mit der ursprünglichen Eingabe verglichen und für jede Variable der quadratische Fehler berechnet, der als Anomalie-Score dient. Dieser Score bildet die Grundlage für die Entscheidung, ob eine Anomalie vorliegt, wobei üblicherweise ein Schwellenwert zur Klassifikation verwendet wird. Im weiteren Verlauf dieses Abschnitts werden verschiedene Varianten des Anomalie-Scores definiert und beschrieben [7].

5.1.1 Anomalie-Score für jede Variable

Der Anomalie-Score für jede einzelne Variable $AS_{\text{var},t,v}$ wird gemäß Gleichung 5.1 berechnet. Dabei bezeichnet $R_{t,v}$ die rekonstruierten Werte, während $W_{t,v}$ die Originalzeitreihe darstellt. Die Dimension der zugrunde liegenden Matrizen beträgt $T \times V$, wobei T die Anzahl der Zeitschritte innerhalb eines Fensters und V die Anzahl der Variablen angibt

[7].

$$AS_{\text{var},t,v} = (R_{t,v} - W_{t,v})^2 \quad (5.1)$$

Für jede Variable muss ein individueller Schwellenwert festgelegt werden, um Anomalien zuverlässig zu erkennen. Dieser Wert dient dazu, zwischen normalem und anomalem Verhalten zu unterscheiden. Im weiteren Verlauf wird dieser Anomalie-Score als punktbasierter Anomalie-Score für jede Variable bezeichnet, da die Bewertung auf der Fehlergröße für jeden einzelnen Datenpunkt basiert.

5.1.2 Anomalie-Score

In vielen Anomalieerkennungssystemen wird der Anomalie-Score über alle Variablen einer Zeitreihe gemittelt, sodass ein einzelner, punktbasierter Anomalie-Score entsteht, der nur noch von der Zeitdimension T abhängt. Dazu wird der Anomalie-Score jeder einzelnen Variable $AS_{\text{var},t,v}$ für jeden Zeitpunkt t über alle V Variablen gemittelt. Die Berechnung erfolgt nach Gleichung 5.2:

$$AS_t = \frac{1}{V} \sum_{v=0}^V AS_{\text{var},t,v} \quad (5.2)$$

Mit diesem Gesamt-Anomalie-Score kann ein einzelner Schwellenwert definiert werden, um zwischen normalen und anomalen Zuständen zu unterscheiden [7]. Dieser Schwellenwert wird für jedes Modell separat ermittelt, sodass nicht nur eine einzelne Metrik maximiert, sondern ein ausgewogenes Verhältnis aller relevanten Metriken ermöglicht wird.

5.1.3 Fensterbasierter Anomalie-Score

Unter der Annahme, dass nur sequentielle Anomalien auftreten, wie es in dieser Arbeit der Fall ist, bietet es sich an, den Anomalie-Score über ein gesamtes Fenster der Größe F zu berechnen. Dieser sogenannte Window-Anomalie-Score (WAS) erlaubt eine robustere Bewertung über Zeiträume hinweg. Das Fenster muss nicht zwingend mit dem Rekonstruktionsfenster des Modells übereinstimmen, was jedoch in dieser Arbeit als Vorgehensweise gewählt wurde.

Das Prinzip ähnelt dem gleitenden Mittelwert, mit dem Unterschied, dass nicht für jeden

Zeitpunkt t des Anomalie-Scores AS_t ein Mittelwert über ein Fenster berechnet wird. Stattdessen genügt es, nur für jeden F -ten Punkt eine Mittelung durchzuführen, wobei F der Fenstergröße entspricht. Der maximale Stride S für die Sliding Windows ist dabei durch $S \leq F$ beschränkt (siehe Abschnitt 4.2.3).

Die Berechnung des fensterbasierten Anomalie-Scores erfolgt nach Gleichung 5.3 für den gesamten Anomalie-Score und nach Gleichung 5.4 für jede einzelne Variable [7]:

$$WAS_t^S = \frac{1}{F} \sum_{i=-\frac{F}{2}}^{\frac{F}{2}} AS_{(t \cdot S) + i} \quad (5.3)$$

$$WAS_t^{S,V} = \frac{1}{F} \sum_{i=-\frac{F}{2}}^{\frac{F}{2}} AS_{\text{var},(t \cdot S) + i, V} \quad (5.4)$$

5.2 Modelle

In diesem Abschnitt werden die verwendeten Modelle zur Anomalieerkennung vorgestellt. Alle Modelle basieren auf der Long Short-Term Memory (LSTM) Architektur, genauer gesagt auf der bidirektionalen Variante (BiLSTM). Diese Architektur ermöglicht es, sowohl vergangene als auch zukünftige zeitliche Abhängigkeiten innerhalb eines Zeitfensters zu erfassen, was insbesondere für die Verarbeitung von sequenziellen Daten vorteilhaft ist.

5.2.1 BiLSTM-AE

Das in dieser Arbeit verwendete BiLSTM-Autoencoder-Modell basiert auf der in der Abbildung 5.1 dargestellten Architektur, wobei einige spezifische Anpassungen vorgenommen wurden, um es optimal für die gegebene Anomalieerkennungsaufgabe zu nutzen. Der Encoder-Teil des Modells besteht aus mehreren bidirektionalen LSTM-Schichten mit abnehmender Größe, wodurch eine zunehmend komprimierte Repräsentation der Eingabedaten erzeugt wird. Dabei wird eine Dropout-Schicht verwendet, um eine bessere Generalisierung des Modells zu gewährleisten. Anschließend wird im Decoder die rekonstruktive Struktur der Architektur genutzt, um die ursprünglichen Eingabedaten aus der komprimierten Repräsentation wiederherzustellen.

Das eigene Modell folgt genau diesem Prinzip und weist eine ähnliche Encoder-Decoder-

Struktur auf. Es verwendet drei bidirektionale LSTM-Schichten im Encoder sowie drei in der Decodierung, die in umgekehrter Reihenfolge angeordnet sind. Zusätzlich wurden Layer-Normalization-Schichten integriert, um eine stabilere Trainingsdynamik zu gewährleisten. Der Ausgang wird über eine TimeDistributed-Dense-Schicht generiert, welche die rekonstruierten Werte für jede Zeitstufe der Eingabe liefert.

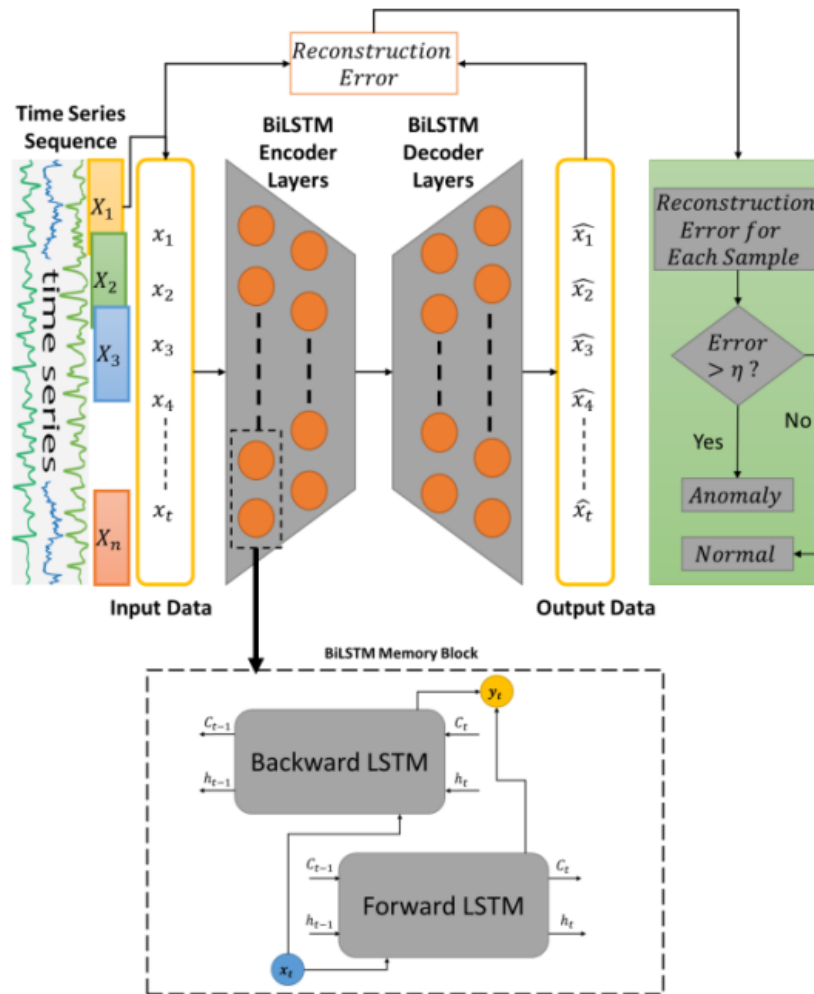


Abbildung 5.1: Schematische Darstellung des BiLSTM-AE Modells [45]

5.2.2 BiLSTM-VAE

Das BiLSTM-VAE-Modell basiert auf der Architektur eines Variational Autoencoders (VAE) und erweitert den zuvor beschriebenen BiLSTM-Autoencoder um eine latente Repräsentation. Während die Encoder- und Decoder-Struktur grundsätzlich der des vorherigen Modells ähnelt, werden anstelle einer festen Repräsentation zwei separate Parameter, die Mittelwerte und Standardabweichungen, für jede latente Dimension berechnet. Dies ermöglicht eine probabilistische Modellierung der Daten, wodurch das

Modell flexibler auf Variationen in der Eingangssequenz reagieren kann.

Die schematische Darstellung in Abbildung 5.2 zeigt den generellen Aufbau eines BiLSTM-VAE. Der Encoder verarbeitet die Eingangszeitreihe durch LSTM-Schichten und projiziert sie in eine latente Verteilung. Aus dieser wird eine latente Variable gesampelt, welche als Eingabe für den Decoder dient. Die Dekodierung erfolgt dann, ähnlich wie beim BiLSTM-AE, durch mehrere LSTM-Schichten, um die ursprüngliche Sequenz zu rekonstruieren.

In der Implementierung wurden drei gestaffelte BiLSTM-Schichten im Encoder genutzt, gefolgt von Dense-Schichten zur Berechnung der latenten Variablen. Die Sampling-Schicht sorgt für die Transformation der latenten Repräsentation, bevor die rekonstruierten Sequenzen im Decoder wiederhergestellt werden. Zusätzlich wurden Layer-Normalization-Schichten integriert, um eine stabilere Trainingsdynamik zu gewährleisten. Der Ausgang wird über eine TimeDistributed-Dense-Schicht generiert, welche die rekonstruierten Werte für jede Zeitstufe der Eingabe liefert.

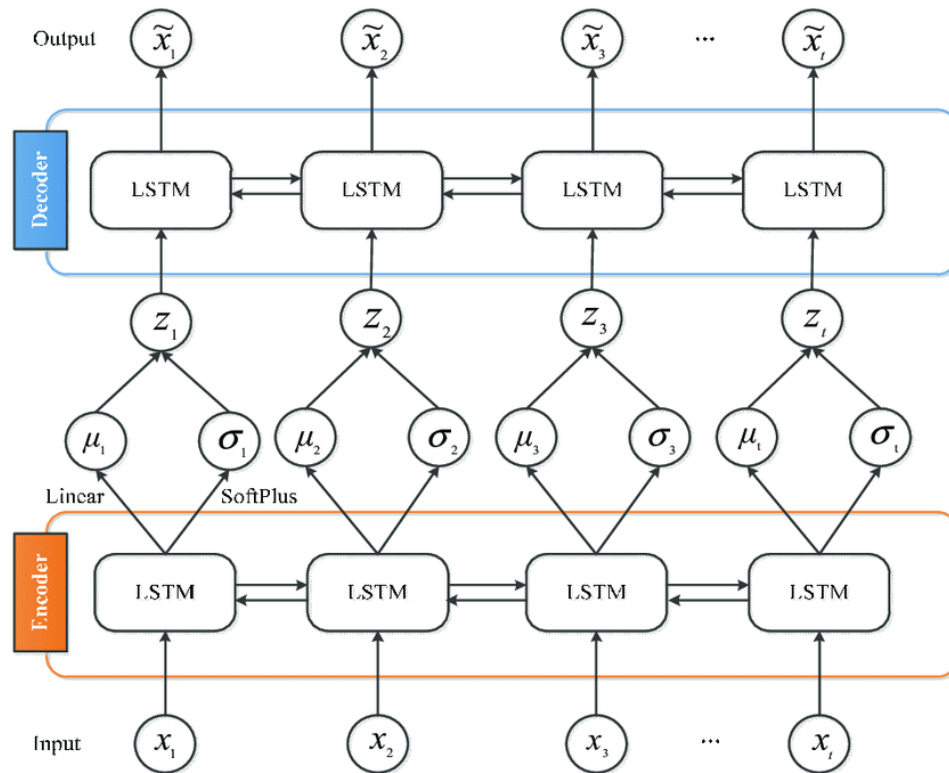


Abbildung 5.2: Schematische Darstellung des BiLSTM-VAE-Modells [46]

5.2.3 Multi-Head Attention-based BiLSTM-AE

Die Kombination von Multi-Head Attention mit BiLSTMs stellt eine leistungsfähige Architektur für die Verarbeitung multivariater Zeitreihen dar. Während BiLSTMs

darauf spezialisiert sind, sequentielle Abhängigkeiten aus vergangenen und zukünftigen Zeitschritten zu modellieren, ergänzt die Multi-Head Attention-Schicht diese Architektur durch eine adaptive Gewichtung relevanter Zeitpunkte. Dadurch kann das Modell verschiedene Merkmale simultan analysieren und komplexe Zusammenhänge innerhalb der Zeitserie besser erfassen [47].

Wie in Abbildung 5.3 dargestellt, beginnt die Verarbeitung mit einer mehrdimensionalen Zeitreihe, die zunächst durch einen BiLSTM-Encoder verarbeitet wird. Anschließend wird die durch das BiLSTM extrahierte Merkmalsdarstellung einer Multi-Head Attention-Schicht zugeführt. Diese Schicht berechnet für jeden Zeitschritt gewichtete Abhängigkeiten zu anderen Zeitpunkten und kombiniert die resultierenden Informationen über mehrere parallele Aufmerksamkeitsmechanismen. Dadurch kann das Modell besonders relevante Zeitpunkte hervorheben und unwichtige oder verrauschte Daten reduzieren, was insbesondere für die Verarbeitung komplexer Zeitreihen vorteilhaft ist [31].

Ein wesentlicher Unterschied zwischen der implementierten Architektur und der in Abbildung 5.3 dargestellten Variante liegt in der zusätzlichen Dense-Schicht, die im implementierten Modell zwischen der Attention-Schicht und dem Decoder eingefügt wurde. Diese Schicht projiziert die latenten Merkmale in einen kompakteren Repräsentationsraum, was insbesondere in autoencoder-basierten Anomalieerkennungsmodellen vorteilhaft ist. Frühere Arbeiten zeigen, dass eine Verdichtung der latenten Merkmale die Robustheit des Modells verbessern und die Trennung zwischen normalen und anomalen Datenpunkten erleichtern kann [48], [49]. Durch diese Projektion wird zudem sichergestellt, dass nur die wesentlichen Merkmale in den Rekonstruktionsprozess einfließen, wodurch der Anomalie-Score eine höhere Aussagekraft erhält.

Zusätzlich wurden Layer-Normalization-Schichten integriert, um eine stabilere Trainingsdynamik zu gewährleisten. Der Ausgang wird über eine TimeDistributed-Dense-Schicht generiert, welche die rekonstruierten Werte für jede Zeitstufe der Eingabe liefert.

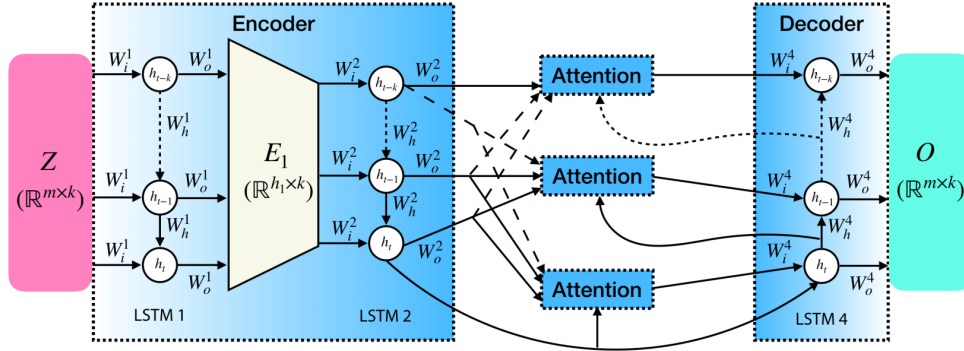


Abbildung 5.3: Architektur eines Multi-Head Attention-based BiLSTM-AE [50]

5.3 Training und Evaluierung

Im Folgenden wird auf das Training der Modelle mit den zwei verschiedenen Datensätzen eingegangen. Die Modelle unterscheiden sich minimal innerhalb des jeweiligen Datensatzes, was jedoch an den Hyperparametern in Kapitel 5.3.3 genauer erläutert wird.

5.3.1 Training

Trainingsdauer

Die Trainingszeit der Modelle kann je nach verwendeten Hardwarekomponenten stark variieren. Für beide Datensätze und alle Modelle wurde eine GeForce RTX 4070 Ti Super von NVIDIA eingesetzt, wobei CUDA zur Hardwarebeschleunigung genutzt wurde. Dies ist wichtig für den späteren Vergleich der Modelle in Abschnitt 7.

TEP-Datensatz

Die Modelle werden ausschließlich mit normalen Daten des TEP-Datensatzes trainiert, um den Normalzustand der Zeitreihen zu erlernen. Die Eingabe besteht aus einem definierten Zeitfenster der Größe F , das die ausgewählten Variablen enthält. Ziel ist es, die ursprünglichen Zeitreihen oder spezifische relevante Variablen möglichst genau zu rekonstruieren. Als Optimierungsverfahren wird Adam verwendet, wobei die Lernrate einem Cosine Decay-Schema folgt. Dabei wird die initiale Lernrate von 1×10^{-3} über 8000 Schritte gemäß einer Cosine-Decay-Funktion mit einem Decay von 0.3 reduziert. Dadurch kann das Modell in frühen Trainingsphasen größere Schritte zur Optimierung unternehmen, während es sich in späteren Epochen feiner anpasst.

Das Training wird mit einer Batchgröße von 64 Batches durchgeführt und läuft über

maximal 100 Epochen. Zur Vermeidung von Overfitting kommt ein Early-Stopping-Mechanismus zum Einsatz, der das Training beendet, wenn sich der Validierungs-Loss über zehn aufeinanderfolgende Epochen nicht verbessert. Dies ermöglicht eine effiziente Optimierung, ohne dass unnötige Epochen durchlaufen werden. Die Verlustfunktion wird mit dem Huber-Loss mit einem δ -Wert von 1.0 verwendet, da dieser sich beim TEP-Datensatz als sehr robust erwiesen hat.

Filling-Station-Datensatz

Auch die Modelle, die auf den Filling-Station-Datensatz trainiert werden, werden ausschließlich mit normalen Daten trainiert, um den Normalzustand zu lernen. Auch hier besteht die Eingabe aus einem definierten Zeitfenster der Größe F , das die gewählten Variablen enthält. Als Optimierungsverfahren wird auch Adam verwendet, wobei β_1 auf 0.9 und β_2 auf 0.98 gesetzt sind. Die Lernrate wird mit Cosine Decay with Warmup durchgeführt. Die Implementierung erfolgt mit einer initialen Lernrate von 5×10^{-4} mit Warm-Up-Schritten, die sich auf vier Mal die Anzahl der Trainingsschritte pro Epoche belaufen. Anschließend wird der Cosine Decay für insgesamt 45 Epochen angewandt.

Abbildung 5.4 zeigt die Lernratenkurve für den Filling-Station-Datensatz.

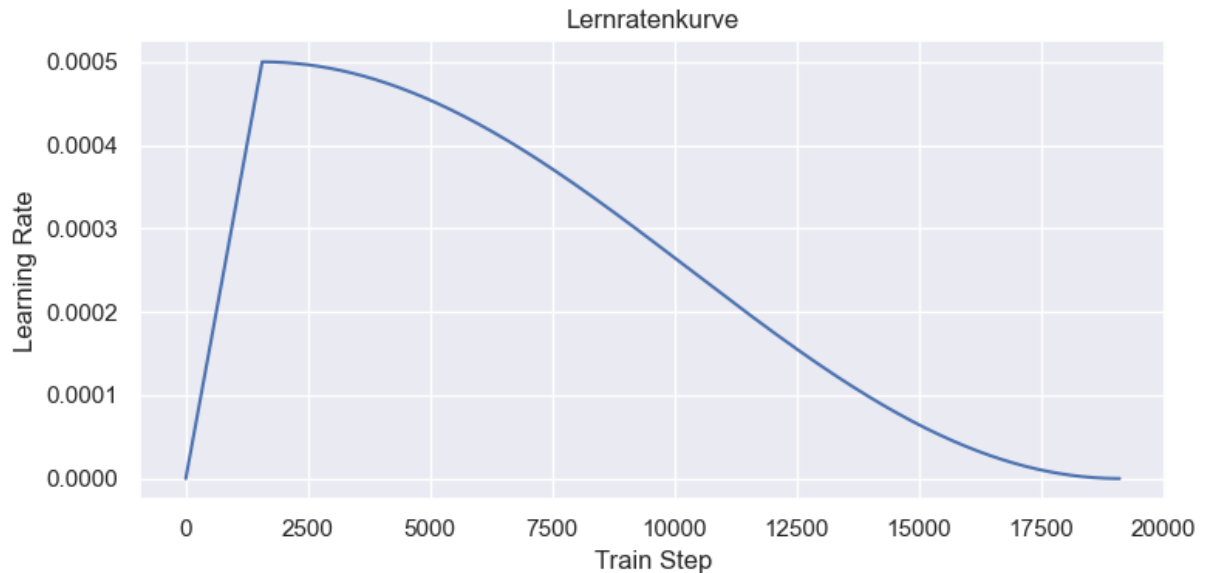


Abbildung 5.4: Verlauf der Lernratenkurve

Das Training wird ebenfalls mit einer Batchgröße von 64 durchgeführt und läuft über maximal 100 Epochen. Zur Vermeidung von Overfitting kommt auch hier ein Early-Stopping-Mechanismus zum Einsatz, der das Training stoppt, wenn sich der Validierungs-

Loss über zehn aufeinanderfolgende Epochen nicht verbessert. Als Verlustfunktion wird hier der Mean Squared Error (MSE) verwendet, da er sich bei diesem Datensatz als sehr robust erwiesen hat.

5.3.2 Evaluierung

Die Validierung des Trainings erfolgt unter Verwendung eines Teils der Normaldaten. Die genaue Aufteilung der Daten ist im jeweiligen Unterkapitel in Abschnitt 4 nachzulesen. Die Auswertung erfolgt durch Analyse der Rekonstruktionen des Modells sowie durch Visualisierung von Beispielen in dieser Arbeit. Die Anomalie-Scores werden in einem Histogramm aufgetragen, um die Trennung zwischen Anomalien und Normalzustand zu erkennen. Die Leistung des Modells wird anhand der Metrik AUCPR (siehe Abschnitt 2.2) in einer Zahl ausgedrückt. Es sei darauf hingewiesen, dass sämtliche in dieser Arbeit angegebenen Ergebnisse den Mittelwert aus mehreren Trainingsdurchläufen darstellen.

5.3.3 Hyperparameter

Die Hyperparameter der einzelnen Modelle wurden empirisch ermittelt. Positive Veränderungen wurden beibehalten und negative revidiert. Die Parameter der Modelle sind innerhalb eines Datensatzes leicht unterschiedlich, im gesamten jedoch ähnlich. Die Parameter der Modelle bei beiden Datensätzen sind jedoch unterschiedlich, wie im vorherigen Abschnitt schon beschrieben. Die genauen Hyperparameter sind in den folgenden Tabellen aufgezeigt:

Hyperparameter	Wert
Epochen	150
Early Stopping	10 Epochen
Batch Größe	64
CosineDecay, Warmup	4 Epochen
CosineDecay, Initial LR	8×10^{-4}
CosineDecay, Decay Steps	8000
CosineDecay, Alpha	0.3
AE-Layout Dimensionen	64, 32, 16
Encoder Dropout Rate	0.2
Decoder Dropout Rate	0.2
Lernbare Parameter	196,676

Tabelle 5.1: Parameter für das BiLSTM-AE Modell mit TEP-Datensatz

Hyperparameter	Wert
Epochen	100
Early Stopping	10 Epochen
Batch Größe	64
CosineDecay, Warmup	4 Epochen
CosineDecay, Initial LR	$4, 5 \times 10^{-4}$
CosineDecay, Decay Steps	45 Epochen
CosineDecay, β_1, β_2	0.9, 0.98
AE-Layout Dimensionen	64, 32, 16
Encoder Dropout Rate	0.3
Decoder Dropout Rate	0.3
Lernbare Parameter	183,235

Tabelle 5.2: Parameter für das BiLSTM-AE Modell mit Filling-Station-Datensatz

Hyperparameter	Wert
Epochen	150
Early Stopping	10 Epochen
Batch Größe	64
CosineDecay, Warmup	4 Epochen
CosineDecay, Initial LR	8×10^{-4}
CosineDecay, Decay Steps	8000
CosineDecay, Alpha	0.3
AE-Layout Dimensionen	64, 32, 16
Latent Dimensionen	16
Encoder Dropout Rate	0.2
Decoder Dropout Rate	0.2
Lernbare Parameter	196,676

Tabelle 5.3: Parameter für das BiLSTM-VAE Modell mit TEP-Datensatz

Hyperparameter	Wert
Epochen	100
Early Stopping	10 Epochen
Batch Größe	64
CosineDecay, Warmup	4 Epochen
CosineDecay, Initial LR	$4,5 \times 10^{-4}$
CosineDecay, Decay Steps	45 Epochen
CosineDecay, β_1, β_2	0.9, 0.98
AE-Layout Dimensionen	64, 32, 16
Latent Dimensionen	16
Encoder Dropout Rate	0.3
Decoder Dropout Rate	0.3
Lernbare Parameter	182,243

Tabelle 5.4: Parameter für das BiLSTM-VAE Modell mit Filling-Station-Datensatz

Hyperparameter	Wert
Epochen	150
Early Stopping	10 Epochen
Batch Größe	64
CosineDecay, Warmup	4 Epochen
CosineDecay, Initial LR	1×10^{-3}
CosineDecay, Decay Steps	8000
CosineDecay, Alpha	0.3
AE-Layout Dimensionen	64, 32, 16
Encoder Dropout Rate	0.2
Decoder Dropout Rate	0.2
Anzahl Heads	5
Key Dimension	32
Attention Dropout Rate	0.2
Lernbare Parameter	217,972

Tabelle 5.5: Parameter für das Attention-based BiLSTM-AE Modell mit TEP-Datensatz

Hyperparameter	Wert
Epochen	100
Early Stopping	10 Epochen
Batch Größe	64
CosineDecay, Warmup	4 Epochen
CosineDecay, Initial LR	$4,5 \times 10^{-4}$
CosineDecay, Decay Steps	45 Epochen
CosineDecay, β_1, β_2	0.9, 0.98
AE-Layout Dimensionen	64, 32, 16
Encoder Dropout Rate	0.3
Decoder Dropout Rate	0.3
Anzahl Heads	4
Key Dimension	32
Attention Dropout Rate	0.2
Lernbare Parameter	196,659

Tabelle 5.6: Parameter für das Attention-based BiLSTM-AE Modell mit Filling-Station-Datensatz

6 Ergebnisse

In diesem Kapitel werden die Ergebnisse der durchgeführten Experimente mit den entwickelten Modellen auf den Datensätzen TEP und Filling-Station vorgestellt. Dabei liegt der Fokus auf ausgewählten Evaluationsmetriken wie AUCPR, F1-Score und MCC, die zur Bewertung der Modelle verwendet wurden.

6.1 TEP-Datensatz

Die folgenden Abbildungen zeigen einen Ausschnitt der Originalzeitreihe zusammen mit der entsprechenden Rekonstruktion durch das Attention-based BiLSTM-AE Modell. Abbildung 6.1 stellt die ursprünglichen Daten im normalen Betriebszustand dar, während Abbildung 6.2 die Originaldaten mit Anomalie und zugehörigem Anomalie-Score aufzeigt.

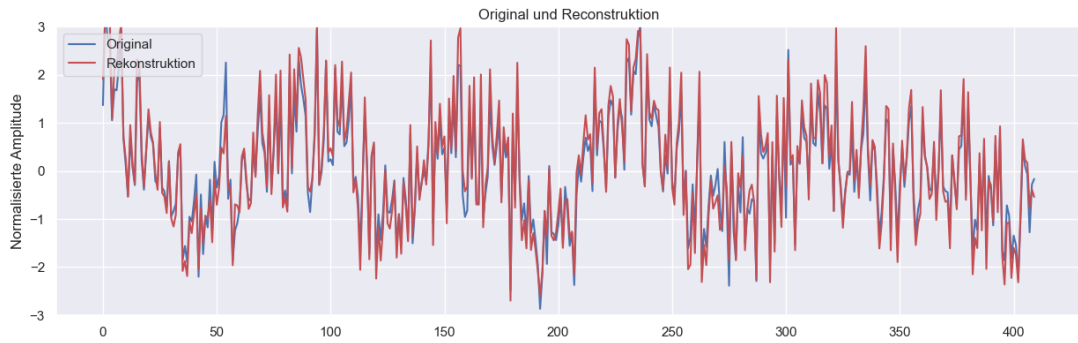


Abbildung 6.1: Originalzeitreihe im Normalzustand mit zugehöriger Rekonstruktion

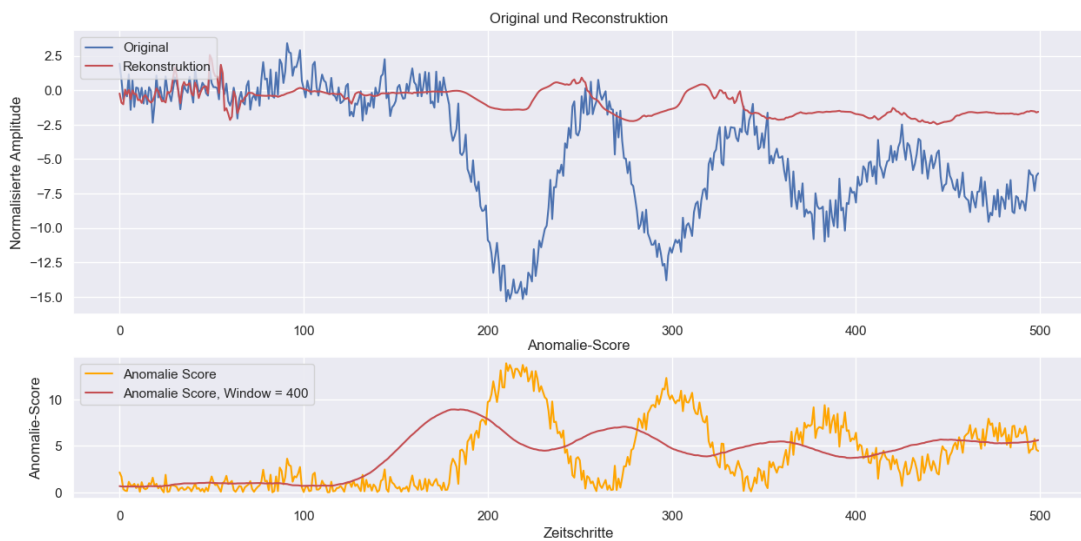


Abbildung 6.2: Originalzeitreihe im anomalen Zustand mit zugehöriger Rekonstruktion und Anomalie-Score

Die Abweichung zwischen der rekonstruierten und der ursprünglichen Zeitreihe ist im anomalen Bereich deutlich ausgeprägt. Dadurch ergibt sich ein hoher Anomalie-Score, der eine zuverlässige Erkennung der Anomalie ermöglicht. Im Gegensatz dazu zeigt die Rekonstruktion im Normalzustand eine hohe Übereinstimmung mit den Originaldaten. Dieses Verhalten ist konsistent über alle angewandten Variablen hinweg zu beobachten.

Die Ergebnisse des AUCPR der drei untersuchten Modelle sind in den nachfolgenden Tabellen zusammengefasst. Jede Tabelle zeigt die entsprechenden Auswertungen für das jeweilige Modell.

Es wird zwischen der klassischen punktbasierten Anomalieerkennung (siehe Abschnitt 5.1.2) und der fensterbasierten Variante (siehe Abschnitt 5.1.3) unterschieden, wobei für Letztere ein Fenster der Größe $F = 400$ verwendet wurde. Bei diesen zwei Werten handelt es sich jeweils um das beste erzielte Ergebnis.

Jedes Modell wird für jede Autoencoder-Struktur mit ihren unterschiedlichen Dimensionen zehnmal trainiert. Die Ergebnisse werden gemittelt und mit ihrer entsprechenden Standardabweichung dargestellt. In Tabelle 6.1 und Tabelle 6.2 sind die Ergebnisse der Modelle einzusehen. Zur Bewertung der Modelleistung werden die Metriken AUCPR, F1-Score, Precision, Recall, Accuracy und MCC verwendet. Der jeweils beste Wert aller Modelle ist fett geschrieben. Der jeweils beste Wert innerhalb des Modells ist kursiv geschrieben.

Für die Berechnung von Precision, Recall und F1-Score wurde pro Modell ein Schwellenwert gewählt, der anhand der Validierungsdaten optimiert wurde.

Der Wert der Trainingsdauer ist mit Vorsicht zu genießen, da mit der Funktion EarlyStopping gearbeitet wurde, um Overfitting beim Training durch zu viele Epochen zu vermeiden. Dementsprechend haben die Modelle zum Teil mit unterschiedlich vielen Epochen trainiert.

Modell	Dimension	AUCPR	F1-Score	Precision	Recall
BiLSTM-AE	16, 8, 4, 4, 8, 16	0.9638 ± 0.0027	0.8826 ± 0.0291	0.9039 ± 0.0270	0.8891 ± 0.0522
	32, 16, 8, 8, 16, 32	0.9656 ± 0.0008	0.8499 ± 0.0213	0.9264 ± 0.0139	0.7881 ± 0.0203
	64, 32, 16, 16, 32, 64	0.9668 ± 0.0007	0.8457 ± 0.0094	0.9418 ± 0.0203	0.7647 ± 0.0295
BiLSTM-VAE	16, 8, 4, 4, 8, 16	0.9539 ± 0.0003	0.7779 ± 0.0006	0.9538 ± 0.0021	0.6570 ± 0.0009
	32, 16, 8, 8, 16, 32	0.9547 ± 0.0006	0.7779 ± 0.0310	0.9538 ± 0.0182	0.6567 ± 0.0421
	64, 32, 16, 16, 32, 64	0.9539 ± 0.0005	0.7780 ± 0.0237	0.9537 ± 0.0113	0.6570 ± 0.0311
Att.-based BiLSTM-AE	16, 8, 4, 4, 8, 16	0.9640 ± 0.0005	0.8326 ± 0.0178	0.9545 ± 0.0127	0.7483 ± 0.0095
	32, 16, 8, 8, 16, 32	0.9694 ± 0.0006	0.8903 ± 0.0381	0.9002 ± 0.0582	0.9311 ± 0.0469
	64, 32, 16, 16, 32, 64	0.9665 ± 0.0004	0.8896 ± 0.0524	0.8581 ± 0.0847	0.8899 ± 0.0674

Tabelle 6.1: Ergebnisse der Modelle, die auf dem TEP-Datensatz trainiert wurden, mit den Metriken: AUCPR, F1-Score, Precision und Recall

Modell	Dimension	Accuracy	MCC	Trainingsdauer
BiLSTM-AE	16, 8, 4, 4, 8, 16	0.7709 ± 0.0325	0.3058 ± 0.0714	$08 : 15 \pm 0 : 47$ min
	32, 16, 8, 8, 16, 32	0.7801 ± 0.0142	0.4669 ± 0.0286	$08 : 37 \pm 0 : 58$ min
	64, 32, 16, 16, 32, 64	0.7855 ± 0.0231	0.4832 ± 0.0172	$10 : 47 \pm 1 : 38$ min
BiLSTM-VAE	16, 8, 4, 4, 8, 16	0.6999 ± 0.0004	0.4257 ± 0.0005	$2 : 37 \pm 0 : 38$ min
	32, 16, 8, 8, 16, 32	0.6999 ± 0.0416	0.4156 ± 0.0263	$5 : 35 \pm 0 : 38$ min
	64, 32, 16, 16, 32, 64	0.7006 ± 0.0331	0.3856 ± 0.0285	$6 : 47 \pm 1 : 20$ min
Att.-based BiLSTM-AE	16, 8, 4, 4, 8, 16	0.7624 ± 0.0101	0.4919 ± 0.0122	$11 : 03 \pm 1 : 01$ min
	32, 16, 8, 8, 16, 32	0.7658 ± 0.0382	0.4750 ± 0.0379	$12 : 36 \pm 0 : 58$ min
	64, 32, 16, 16, 32, 64	0.8033 ± 0.0431	0.4142 ± 0.0854	$13 : 47 \pm 1 : 13$ min

Tabelle 6.2: Ergebnisse der Modelle, die auf dem TEP-Datensatz trainiert wurden, mit den Metriken: Accuracy, MCC und Trainingsdauer

Für die bessere Übersicht, sind die AUCPR und F1-Score-Werte der drei Modelle, in den Boxplots 6.3 und 6.4 dargestellt.

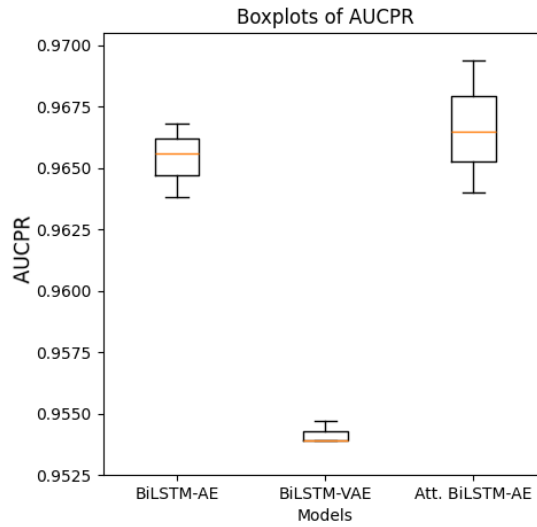


Abbildung 6.3: Boxplot des AUCPR Wertes des TEP-Datensatzes

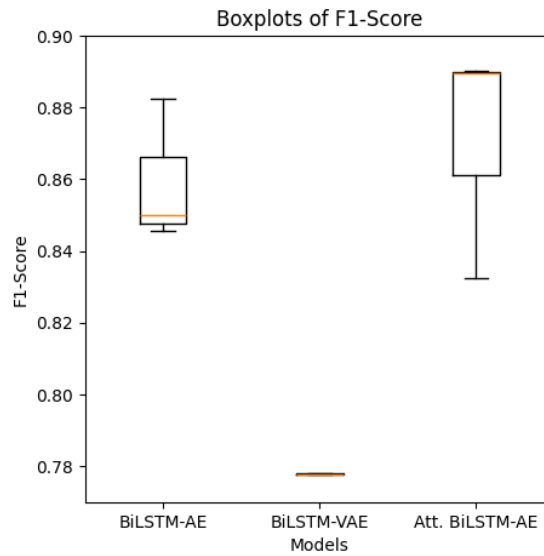


Abbildung 6.4: Boxplot des F1-Score-Wertes des TEP-Datensatzes

6.2 Filling-Station-Datensatz

Die folgenden Abbildungen zeigen einen Ausschnitt der Originalzeitreihe und der Rekonstruktion dieser mit dem Attention-based BiLSTM-AE Modell. In Abbildung 6.5 ist ein Beispiel im Normalzustand zu sehen, während Abbildung 6.6 und Abbildung 6.7 einen anomalen Zustand sowie den zugehörigen Anomalie-Score abbilden.

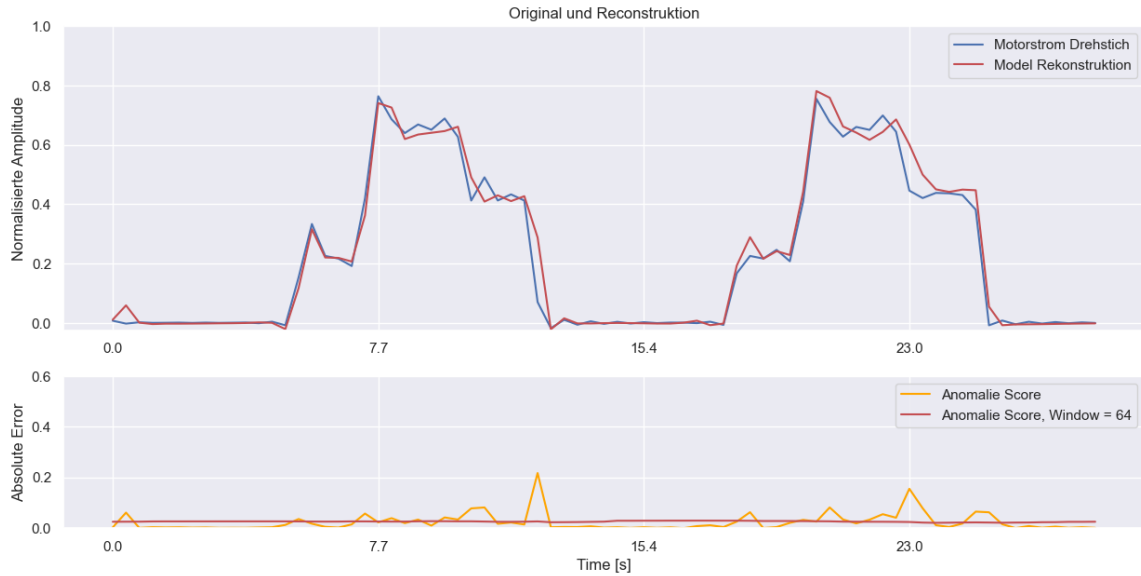


Abbildung 6.5: Modellrekonstruktion und Anomalie-Score der Variable Motorstrom am Drehtisch ohne Anomalie

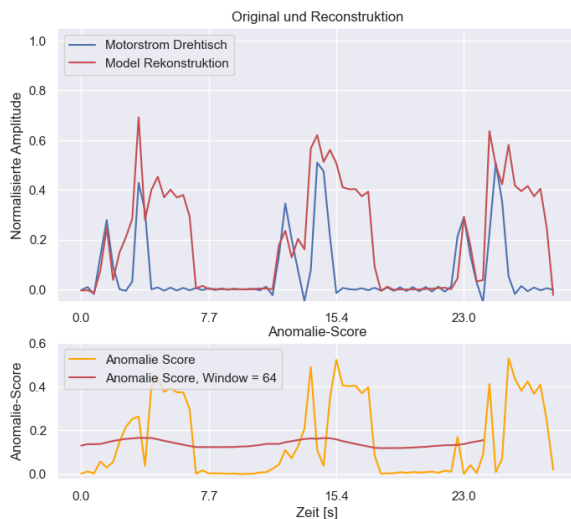


Abbildung 6.6: Modellrekonstruktion und Anomalie-Score der Variable Motorstrom am Drehtisch mit der Anomalie „zu wenig Widerstand“

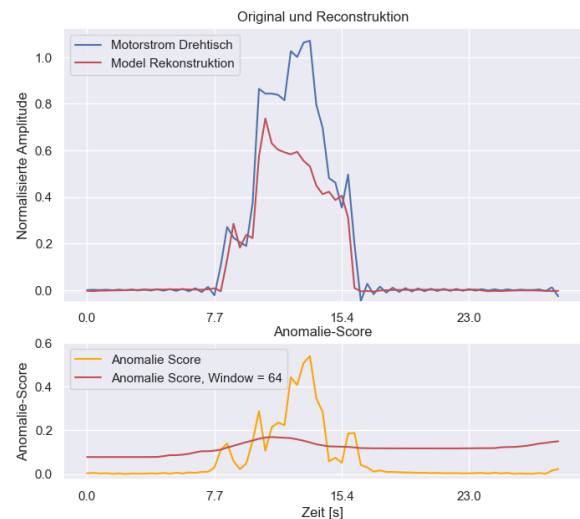


Abbildung 6.7: Modellrekonstruktion und Anomalie-Score der Variable Motorstrom am Drehtisch mit der Anomalie „zu viel Widerstand“

Bei den anomalen Daten ist deutlich zu erkennen, dass die Rekonstruktion deutlich von

der ursprünglichen Zeitreihe abweicht. Dadurch ergibt sich ein erhöhter Anomalie-Score, der eine zuverlässige Identifikation der Anomalie ermöglicht. Im Gegensatz dazu stimmt die Rekonstruktion im Normalzustand weitgehend mit den Originaldaten überein.

Da die Anomalien im Datensatz nicht durchgängig anomal sind, wird ausschließlich der fensterbasierte Anomalie-Score verwendet (vgl. Abschnitt 5.1.3). Das betrachtete Fenster hat die gleiche Größe wie bei der Rekonstruktion ($F = 64$).

Jedes Modell wird für jede Autoencoder Struktur mit ihren unterschiedlichen Dimensionen zehnmal trainiert. Die Ergebnisse werden gemittelt mit ihrer entsprechenden Standardabweichung dargestellt. In Tabelle 6.3 und Tabelle 6.4 sind die Ergebnisse der Modelle einzusehen. Zur Bewertung der Modellleistung werden die gleichen Metriken wie beim TEP-Datensatz verwendet. Der jeweils beste Wert aller Modelle ist fett geschrieben. Der jeweils beste Wert innerhalb des Modells ist kursiv geschrieben.

Für die Berechnung von Precision, Recall und F1-Score wurde pro Modell ein Schwellenwert gewählt, der anhand der Validierungsdaten optimiert wurde.

Auch hier ist wie beim TEP-Datensatz der Wert der Trainingsdauer mit Vorsicht zu genießen, da ebenfalls mit der Funktion EarlyStopping gearbeitet wurde, um Overfitting beim Training durch zu viele Epochen zu vermeiden. Dementsprechend haben die Modelle zum Teil mit unterschiedlich vielen Epochen trainiert.

Modell	Dimension	AUCPR	F1-Score	Precision	Recall
BiLSTM-AE	16, 8, 4, 4, 8, 16	0.9958 ± 0.0034	0.9761 ± 0.0130	0.9708 ± 0.0091	0.9854 ± 0.0146
	32, 16, 8, 8, 16, 32	0.9920 ± 0.0063	0.9563 ± 0.0166	0.9624 ± 0.0161	0.9615 ± 0.0384
	64, 32, 16, 16, 32, 64	0.9672 ± 0.0084	0.9143 ± 0.0236	0.9474 ± 0.0310	0.8967 ± 0.0092
BiLSTM-VAE	16, 8, 4, 4, 8, 16	0.7883 ± 0.0341	0.8230 ± 0.0101	0.7646 ± 0.0511	0.8853 ± 0.0246
	32, 16, 8, 8, 16, 32	0.9886 ± 0.0075	0.9445 ± 0.0122	0.9711 ± 0.0229	0.9380 ± 0.0328
	64, 32, 16, 16, 32, 64	0.9551 ± 0.0371	0.8943 ± 0.0211	0.9031 ± 0.0210	0.8957 ± 0.0232
Att.-based BiLSTM-AE	16, 8, 4, 4, 8, 16	0.9955 ± 0.0039	0.9792 ± 0.0192	0.9787 ± 0.0200	0.9742 ± 0.0248
	32, 16, 8, 8, 16, 32	0.9965 ± 0.0035	0.9892 ± 0.0108	0.9887 ± 0.0113	0.9942 ± 0.0058
	64, 32, 16, 16, 32, 64	0.9822 ± 0.0161	0.9492 ± 0.0167	0.9326 ± 0.0229	0.9566 ± 0.0232

Tabelle 6.3: Ergebnisse der Modelle, die auf dem Filling-Station-Datensatz trainiert wurden, mit den Metriken: AUCPR, F1-Score, Precision und Recall

Modell	Dimension	Accuracy	MCC	Trainingsdauer
BiLSTM-AE	16, 8, 4, 4, 8, 16	0.9726 ± 0.0140	0.9591 ± 0.0226	13 : 21 $\pm$ 0 : 32 min
	32, 16, 8, 8, 16, 32	0.9775 ± 0.0104	0.9320 ± 0.0274	14 : 04 $\pm$ 0 : 43 min
	64, 32, 16, 16, 32, 64	0.9273 ± 0.0198	0.8561 ± 0.431	14 : 09 $\pm$ 0 : 23 min
BiLSTM-VAE	16, 8, 4, 4, 8, 16	0.8327 ± 0.0352	0.6932 ± 0.0493	13 : 26 $\pm$ 0 : 41 min
	32, 16, 8, 8, 16, 32	0.9552 ± 0.0086	0.9171 ± 0.0237	14 : 45 $\pm$ 0 : 28 min
	64, 32, 16, 16, 32, 64	0.9149 ± 0.0351	0.8233 ± 0.0612	18 : 17 $\pm$ 0 : 41 min
Att.-based BiLSTM-AE	16, 8, 4, 4, 8, 16	0.9844 ± 0.0141	0.9661 ± 0.0316	14 : 05 $\pm$ 0 : 20 min
	32, 16, 8, 8, 16, 32	0.9944 ± 0.0056	0.9821 ± 0.0179	14 : 43 $\pm$ 0 : 45 min
	64, 32, 16, 16, 32, 64	0.9486 ± 0.0294	0.8979 ± 0.0454	16 : 15 $\pm$ 0 : 38 min

Tabelle 6.4: Ergebnisse der Modelle, die auf dem Filling-Station-Datensatz trainiert wurden, mit den Metriken: Accuracy, MCC und Trainingsdauer

Für die bessere Übersicht sind die AUCPR und F1-Score-Werte der drei Modelle in den Boxplots 6.8 und 6.9 dargestellt.

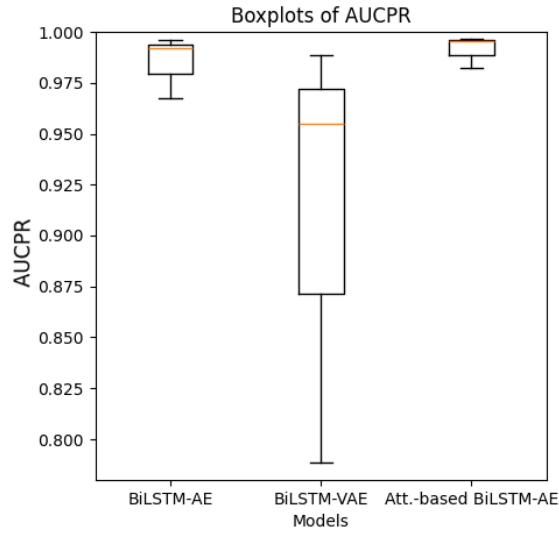


Abbildung 6.8: Boxplot des AUCPR Wertes des Filling-Station-Datensatzes

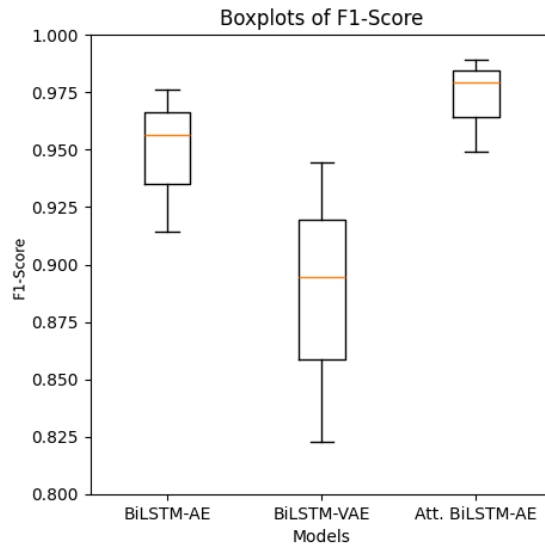


Abbildung 6.9: Boxplot des F1-Score-Wertes des Filling-Station-Datensatzes

7 Diskussion

In diesem Kapitel werden die Ergebnisse aus Kapitel 6 interpretiert. Es wird auf die Stärken und Schwächen der Modelle eingegangen, mögliche Ursachen für die Unterschiede in den Metriken diskutiert und Verbesserungspotenziale aufgezeigt.

7.1 TEP-Datensatz

Die Ergebnisse in den Tabellen 6.1 und 6.2 sowie die Boxplots (Abbildungen 6.3 und 6.4) geben interessante Einblicke in die Leistung der verschiedenen Modelle auf dem TEP-Datensatz. Zunächst ist festzustellen, dass die Modelle im Hinblick auf den AUCPR-Wert insgesamt gut abschneiden, wobei das Attention-based BiLSTM-AE-Modell mit einem Durchschnitt von 0.9665 den höchsten AUCPR-Wert erzielt, gefolgt vom BiLSTM-AE-Modell (0.9656). Das BiLSTM-VAE-Modell zeigt mit einem Durchschnitt von 0.9539 den niedrigeren Wert im Vergleich zu den anderen beiden Modellen.

Die Boxplots in Abbildung 6.3 veranschaulichen zusätzlich die Verteilung der AUCPR-Werte über mehrere Trainingsläufe hinweg und ermöglichen somit eine Aussage über die Stabilität der Modelle. Dabei zeigt sich, dass das BiLSTM-VAE-Modell die geringste Streuung aufweist, was auf eine hohe Reproduzierbarkeit und Robustheit der Ergebnisse schließen lässt. Im Gegensatz dazu weisen das BiLSTM-AE-Modell und insbesondere das Attention-based BiLSTM-AE-Modell eine deutlich größere Streuung auf. Dies deutet darauf hin, dass diese Modelle sensibler auf Initialisierungen oder Trainingsschwankungen reagieren und ihre Leistung stärker von einzelnen Trainingsläufen abhängt. Eine mögliche Ursache dafür ist die höhere Komplexität dieser Modelle, insbesondere beim Attention-Mechanismus, der mehr Parameter und damit potenziell höhere Varianz im Training mit sich bringt.

Ein signifikanter Einfluss auf die hohen AUCPR-Werte der Modelle, jedoch auf die schlechten Werte der anderen Metriken, könnte die ungleich verteilte Testdatenbasis sein. Anomalien treten im Datensatz 20-mal häufiger auf als normale Daten, was dazu führt, dass selbst einfache Modelle mit begrenztem Wissen über den Normalzustand hohe AUCPR-Werte erzielen. Eine kleine Änderung des AUCPR-Wertes hat daher signifikante Auswirkungen auf die Erkennungsrate. Dies spiegelt sich in der kleinen Standardabweichung

des AUCPR-Wertes wider, während die restlichen Werte große Standardabweichungen aufweisen.

Ein auffälliges Detail in den Ergebnissen ist, dass das BiLSTM-VAE-Modell trotz seiner geringen Streuung der AUCPR-Werte in den übrigen Metriken wie Precision und Recall nicht so gut abschneidet, wie die anderen beiden Modelle. Dies könnte darauf hindeuten, dass die Architektur des BiLSTM-VAE-Modells grundsätzlich weniger gut geeignet ist, die im TEP-Datensatz auftretenden Anomalien zuverlässig zu erfassen, obwohl sie konsistente Ergebnisse liefert.

Der F1-Score (Abbildung 6.4) unterstützt diese Beobachtungen, indem er das Attention-based BiLSTM-AE-Modell mit einem Durchschnitt von 0.8903 an die Spitze setzt, was auf eine hervorragende Balance zwischen Precision und Recall hinweist. Das BiLSTM-AE-Modell schneidet mit einem Durchschnitt von 0.8499 ebenfalls gut ab, jedoch ist der Unterschied zum Attention-based BiLSTM-AE-Modell im Hinblick auf den F1-Score deutlich sichtbar. Das BiLSTM-VAE-Modell hingegen zeigt den niedrigsten Wert von 0.7779, was seine im Vergleich schlechtere Leistung bei der Anomalieerkennung weiter bestätigt.

Die Tabellen und Diagramme legen nahe, dass die Wahl der Modellarchitektur einen wesentlichen Einfluss auf die Leistungsfähigkeit der Modelle hat. Insbesondere das Attention-based BiLSTM-AE-Modell erzielt die besten Ergebnisse in beiden Metriken (AUCPR und F1-Score), was auf seine Fähigkeit hinweist, sowohl die Genauigkeit als auch die Recall-Rate zu optimieren. Die höhere Komplexität dieses Modells trägt dazu bei, dass es in der Lage ist, feinere Muster in den Anomalien zu erkennen.

Zusammenfassend lässt sich sagen, dass das BiLSTM-VAE-Modell in Bezug auf die Anomalieerkennung im TEP-Datensatz hinter den anderen beiden Modellen zurückbleibt, was vor allem an seiner Architektur und der Fähigkeit, zeitliche Abhängigkeiten zu modellieren, liegen könnte. Das Attention-based BiLSTM-AE-Modell zeigt sich hingegen als die robusteste Lösung für diese Aufgabe, da es sowohl in der AUCPR als auch im F1-Score deutlich besser abschneidet als die anderen Modelle.

7.1.1 Einfluss des Klassenungleichgewichts auf den MCC-Wert

Ein signifikantes Problem in der Modellbewertung bei unbalancierten Datensätzen, wie im vorliegenden Fall, ist die Verzerrung von Evaluationsmetriken wie dem MCC, die die Balance zwischen den Fehlerarten (True Positives, True Negatives, False Positives und False Negatives) berücksichtigen. Im vorliegenden Fall, bei dem Anomalien 20-mal häufiger als normale Daten auftreten, sind die Modellmetriken wie AUCPR und F1-Score zwar insgesamt hoch, jedoch leidet der MCC-Wert unter den Auswirkungen dieses Ungleichgewichts. Ein niedriger MCC-Wert trotz gutem Abschneiden in den anderen Metriken kann darauf hindeuten, dass das Modell Schwierigkeiten hat, die Anomalien korrekt zu klassifizieren.

In unseren Ergebnissen zeigt sich, dass der MCC-Wert, insbesondere bei Modellen wie dem BiLSTM-VAE, signifikant niedriger ist als bei den anderen Modellen, obwohl die anderen Metriken wie AUCPR und Precision eine gute Leistung anzeigen. Dies ist ein klares Zeichen dafür, dass das Modell bei der Klassifikation der Anomalien schwächelt, was sich negativ auf den MCC auswirkt. Der MCC-Wert von 0.13, der für das BiLSTM-VAE-Modell erzielt wurde, deutet darauf hin, dass das Modell die weniger häufigen Anomalien nicht ausreichend korrekt klassifiziert, obwohl es in anderen Bereichen gut abschneidet.

Dieser Trend wird durch die hohe Anzahl an False Positives und False Negatives verstärkt, die sich negativ auf den MCC auswirken, während die anderen Metriken weniger empfindlich auf diese Fehlerarten reagieren. Der niedrige MCC-Wert zeigt daher, dass, auch wenn das Modell in der Erkennung der Mehrheit der normalen Daten gut ist, es bei den selteneren Anomalien versagt, was sich in den anderen Metriken nicht immer widerspiegelt.

Ein niedriger MCC, trotz hoher Werte in anderen Metriken, hebt hervor, wie wichtig es ist, alle vier Fehlerarten bei der Modellbewertung zu berücksichtigen, insbesondere bei unbalancierten Datensätzen. Die Ergebnisse verdeutlichen, dass der MCC ein umfassenderes Bild der Modelleleistung liefert und vor allem auf Probleme bei der Klassifikation der Anomalien hinweist, die durch das Klassenungleichgewicht verstärkt werden.

7.1.2 Vergleich mit Modellen aus der Literatur

In der Publikation von Hartung et al. [51] wurden zahlreiche Modelle auf dem TEP-Datensatz untersucht. Ein Auszug der Ergebnisse ist in Tabelle 7.1 dargestellt. Die eigenen

Modelle wurden in die Tabelle hinzugefügt und sind kursiv geschrieben.

Modell	AUCPR	F1-Score
TCN-S2S-AE	0.9914	0.9632
BeatGAN	0.9896	0.9699
OmniAnomaly	0.9808	0.9336
Untrainiertes LSTM-AE	0.9792	0.9333
<i>Attention-based BiLSTM-AE</i>	0.9694 ± 0.0006	0.8903 ± 0.0381
<i>BiLSTM-AE</i>	0.9668 ± 0.0007	0.8457 ± 0.0094
Mad-GAN	0.9621	0.9333
<i>BiLSTM-VAE</i>	0.9539 ± 0.0003	0.7779 ± 0.0006
LSTM-2S2-P	0.9171	0.9327

Tabelle 7.1: Vergleich der besten Modelle aus dieser Arbeit mit den Ergebnissen aus [51] auf Basis der Metriken AUCPR und F1-Score

Die Modelle aus dieser Arbeit zeigen AUCPR-Werte, die sich im unteren Bereich des Spektrums im Vergleich zu den Top-Modellen aus der Publikation von Hartung et al. [51] bewegen. Insbesondere die BiLSTM-AE- und Attention-based BiLSTM-AE-Modelle erzielen AUCPR-Werte von 0.96 bis 0.97, was zwar ansehnlich ist, jedoch unter den Ergebnissen von Modellen wie dem TCN-S2S-AE (0.9914 AUCPR) und BeatGAN (0.9896 AUCPR) bleibt.

Das BiLSTM-VAE-Modell erreicht noch niedrigere AUCPR-Werte von rund 0.95, was möglicherweise durch die Architektur des VAE-Modells bedingt ist, die Schwierigkeiten bei der Modellierung der zeitlichen Abhängigkeiten und der Anomalien haben könnte. Obwohl das BiLSTM-VAE-Modell mit 0.95 immer noch solide Ergebnisse liefert, erreicht es nicht die Spitzenwerte der anderen Modelle.

Im Vergleich zu den anderen Modellarchitekturen aus der Veröffentlichung von Hartung et al. fällt besonders auf, dass die Attention-based BiLSTM-AE-Modelle bei AUCPR und F1-Score tendenziell besser abschneiden als die BiLSTM-AE- und BiLSTM-VAE-Modelle. Dies deutet darauf hin, dass die zusätzliche Aufmerksamkeitsschicht im Attention-based BiLSTM-AE-Modell die Anomalieerkennung verbessert, jedoch nicht auf das Niveau der besten Modelle.

In der Arbeit von Schoch, 2024 [7] wird ein Transformer-Modell vorgestellt, das sich ebenfalls gut im Vergleich macht. Mit einem AUCPR-Wert und einem F1-Score von jeweils

0.9954 übertrifft dieses Modell das TCN-S2S-AE-Modell. Besonders hervorzuheben ist, dass Transformer-Modelle ausschließlich auf Attention-Mechanismen basieren, was die Effektivität von Attention-based Modellarchitektur weiter unterstreicht.

Insgesamt bewegen sich die Modelle aus dieser Arbeit im mittleren Bereich der Modelle aus der Literatur, was die Effektivität der entwickelten Methoden zur Anomalieerkennung bestätigt. Sie bieten solide und konsistente Ergebnisse, sind jedoch noch nicht in der Lage, mit den leistungsfähigeren Modellen zu konkurrieren. Die Modelle in dieser Arbeit zeichnen sich dennoch durch eine vergleichsweise geringe Modellkomplexität aus, was sie für Anwendungen in Echtzeit und für ressourcenbeschränkte Szenarien in der Industrie attraktiv macht.

7.2 Filling-Station-Datensatz

Die Ergebnisse in Tabelle 6.3 und 6.4 sowie die Boxplots (Abbildungen 6.8 und 6.9) bieten eine detaillierte Übersicht über die Leistung der verschiedenen Modelle auf dem Filling-Station-Datensatz. Zu beachten ist, dass die Modelle insgesamt im Hinblick auf den AUCPR-Wert vergleichbare Ergebnisse erzielen, jedoch in den anderen Metriken, insbesondere im F1-Score, Unterschiede bestehen.

Bezüglich des AUCPR-Werts erzielt das Attention-based BiLSTM-AE-Modell mit einem Durchschnitt von 0.9955 die höchste Leistung, gefolgt vom BiLSTM-AE-Modell mit einem Wert von 0.9920. Das BiLSTM-VAE-Modell zeigt mit einem Durchschnitt von 0.9551 einen deutlich niedrigeren Wert, was auf die Unterschiede in der Architektur hinweist und darauf, dass das Modell Schwierigkeiten bei der Modellierung der Anomalien im Vergleich zu den anderen beiden hat.

Die Boxplots verdeutlichen diese Ergebnisse, da die Verteilung der AUCPR-Werte beim BiLSTM-VAE-Modell eine größere Streuung aufweist, während BiLSTM-AE und Attention-based BiLSTM-AE eine stabilere Leistung zeigen. Dies könnte darauf hinweisen, dass die Modelle mit der Attention-Schicht und das BiLSTM-AE robuster im Umgang mit dem Datensatz sind und weniger Schwankungen bei wiederholten Trainingsläufen auftreten.

Im Hinblick auf den F1-Score zeigt das Attention-based BiLSTM-AE-Modell mit einem Durchschnitt von 0.9792 die beste Leistung. Das BiLSTM-AE-Modell schneidet mit einem Durchschnitt von 0.9708 ebenfalls gut ab, bleibt aber hinter dem Attention-Modell zurück.

Das BiLSTM-VAE-Modell erzielt einen deutlich niedrigeren Wert von 0.8230, was auf eine schlechtere Balance zwischen Precision und Recall hinweist. Die hohe Streuung des BiLSTM-VAE-Modells im F1-Score bestätigt diese Beobachtungen, da der Wert im Vergleich zu den anderen Modellen stärkeren Schwankungen unterliegt.

Zusammenfassend lässt sich sagen, dass das Attention-based BiLSTM-AE-Modell sowohl im AUCPR als auch im F1-Score die besten Ergebnisse erzielt. Dies könnte auf die Verbesserung der Anomalieerkennung durch die Attention Heads hinweisen. Das BiLSTM-AE-Modell liefert ebenfalls solide Ergebnisse, jedoch in einer geringeren Leistungsbandbreite als das Attention-based Modell. Das BiLSTM-VAE-Modell schneidet in beiden Metriken etwas schwächer ab, was auf die Architektur und mögliche Schwierigkeiten bei der Modellierung von Anomalien zurückzuführen ist.

Die Wahl des Modells und der Architektur hat einen wesentlichen Einfluss auf die Leistung. Es zeigt sich, dass das Attention-based BiLSTM-AE-Modell besonders geeignet ist, die Komplexität des Filling-Station-Datensatzes zu erfassen und gleichzeitig eine stabile und konsistente Leistung zu liefern.

7.3 Zusammenführung der Diskussionsergebnisse

Das Attention-based BiLSTM-Autoencoder-Modell konnte die besten Ergebnisse bei der Anomalieerkennung erzielen. Es konnte wichtige Zeitpunkte besonders gut gewichten und dadurch ein gutes Gleichgewicht zwischen Precision und Recall erreichen. Das klassische BiLSTM-AE war ebenfalls leistungsfähig, während das BiLSTM-VAE in beiden Datensätzen deutlich schlechter abschnitt, vor allem bei der Rekonstruktionsgenauigkeit. Außerdem wurde deutlich, dass eine ungleichmäßige Verteilung von Normal- und Anomaliedaten die Bewertung einzelner Metriken beeinflusste. Deshalb waren zusätzliche Kennzahlen wie der MCC hilfreich. Insgesamt lieferten die getesteten Modelle gute Ergebnisse und eigneten sich gut für einfache, ressourcenschonende Anwendungen in der Industrie.

8 Zusammenfassung und Ausblick

8.1 Zusammenfassung

In Abschnitt 1 wurden die Grundlagen und Ziele dieser Arbeit vorgestellt. Dabei wurde erläutert, welche Relevanz die Anomalieerkennung in industriellen Prozessen besitzt und warum rekurrente neuronale Netze in Kombination mit Autoencoder-Architekturen vielversprechend sind. Zudem wurde der Aufbau der Arbeit dargelegt.

In Abschnitt 2 wurden die theoretischen Grundlagen beschrieben, die für das Verständnis der weiteren Abschnitte von Bedeutung sind. Dazu gehörten Methoden zur Anomalieerkennung, relevante Bewertungsmetriken sowie die Konzepte rekurrenter Netze und die Idee der Attention Heads. Zusätzlich wurde die Modellfabrik als Beispiel einer industriellen Anlage vorgestellt.

Abschnitt 3 präsentierte einige bereits existierende Arbeiten im Bereich der Anomalieerkennung und gab damit den aktuellen Forschungsstand wieder. Dabei zeigte sich, dass insbesondere tiefe und rekurrente Netzwerke für Zeitreihenanalysen verbreitet sind, jedoch auch Transformermodelle und Attention-Mechanismen eine zunehmend wichtige Rolle spielen.

In Abschnitt 4 wurden die verwendeten Datenquellen erläutert. Neben dem bekannten Tennessee Eastman Process Datensatz kam ein aus der Modellfabrik aufgezeichneter Datensatz zum Einsatz. Es wurde beschrieben, wie die Daten aufgenommen und vorverarbeitet wurden, um sie für das Training neuronaler Netze zu nutzen. Dadurch standen sowohl ein synthetischer als auch ein realer Datensatz zur Verfügung.

Abschnitt 5 behandelte die Methodik der Arbeit. Dort wurde erklärt, welche Modelle eingesetzt und wie sie im Detail aufgebaut waren. Im Mittelpunkt standen BiLSTM-Autoencoder-, BiLSTM-Variational-Autoencoder- und Attention-based BiLSTM-Modelle. Ebenfalls wurde die Vorgehensweise bei der Anomalieerkennung sowie die Evaluationsmethodik beschrieben.

Die in Abschnitt 6 präsentierten Ergebnisse zeigten die Leistungsfähigkeit der Modelle bei der Anomalieerkennung. Der Fokus lag auf Kennzahlen wie AUCPR und F1-Score, um einen umfassenden Vergleich zu ermöglichen. Für beide Datensätze wurden die Resultate

tabellarisch und grafisch dargestellt.

Anschließend erfolgte in Abschnitt 7 eine Diskussion der gewonnenen Erkenntnisse. Stärken und Schwächen der Ansätze wurden herausgearbeitet, um zu verdeutlichen, unter welchen Bedingungen welche Modelle besonders gut oder weniger gut funktionierten. Hier wurde deutlich, dass die Wahl der Modellarchitektur sowie die Beschaffenheit der Daten einen großen Einfluss auf die Erkennungsleistung hatten.

Insgesamt konnte in dieser Arbeit aufgezeigt werden, dass BiLSTM-based Autoencoder-Varianten, insbesondere bei längeren oder komplex korrelierten Zeitreihen, gute Ergebnisse liefern und dass ein zusätzlicher Attention-Mechanismus die Leistung weiter steigern kann. Dank des Einsatzes eines realen Datensatzes konnte zudem veranschaulicht werden, dass diese Konzepte praxisnah umsetzbar sind und einen wertvollen Beitrag zur automatisierten Anomalieerkennung in industriellen Prozessen leisten.

8.2 Kritische Bewertung der Ergebnisse

Die Ergebnisse der drei Modelle zeigen deutliche Unterschiede in ihrer Leistungsfähigkeit, die bei der Bewertung differenziert betrachtet werden müssen. Dabei fällt auf, dass sich das Verhalten der Modelle zwischen den beiden verwendeten Datensätzen zum Teil deutlich unterscheidet.

Das BiLSTM-VAE-Modell zeigte ein inkonsistentes Verhalten über die beiden Datensätze hinweg. Während es im TEP-Datensatz durch eine sehr geringe Streuung der AUCPR-Werte auffiel und damit eine hohe Stabilität suggerierte, wies es im Filling-Station-Datensatz eine deutlich höhere Varianz auf – bei gleichzeitig insgesamt schwächeren Metriken wie Precision, Recall und F1-Score. Diese starke Streuung deutet auf eine ausgeprägte Sensitivität gegenüber Trainingsinitialisierungen oder Modellparametern hin. Insgesamt bleibt das Modell hinter den anderen Ansätzen zurück, was vermuten lässt, dass die gewählte Latentraumstruktur nicht gut geeignet ist, um die relevanten Muster der Anomalien robust zu erfassen. In der Literatur ist dieses Verhalten bekannt: VAEs neigen dazu, feine Unterschiede zwischen Normal- und Anomaliedaten zu verwischen, insbesondere wenn der latente Raum durch eine zu starke Regularisierung geglättet wird [52], [53].

Das klassische BiLSTM-AE-Modell schnitt in beiden Datensätzen insgesamt solide ab,

zeigte jedoch eine merkbliche Streuung über verschiedene Trainingsläufe hinweg. Die Leistung war stark abhängig von den Trainingsbedingungen, was sich insbesondere bei schwieriger zu erkennenden Anomalien in einem teils deutlich schwankenden Recall widerspiegelte. Zwar wurden gute AUCPR-Werte erreicht, dennoch offenbart die Varianz im F1-Score eine gewisse Unsicherheit in der Modellstabilität. Diese Instabilität war im TEP-Datensatz stärker ausgeprägt, während sich im Filling-Station-Datensatz eine etwas konsistentere Leistung zeigte. Ähnliche Beobachtungen finden sich in der Literatur, wo LSTM-basierte Autoencoder bei langen Zeitreihen durch Fehlerakkumulation oder Konvergenzprobleme in ihrer Robustheit eingeschränkt sind [54].

Das Attention-based Modell lieferte in beiden Datensätzen die besten Metriken, insbesondere beim F1-Score und AUCPR-Wert. Hinsichtlich der Stabilität zeigten sich jedoch deutliche Unterschiede: Während das Modell im TEP-Datensatz die größte Streuung zwischen den Trainingsläufen aufwies, war die Varianz im Filling-Station-Datensatz deutlich geringer. Dieses Verhalten lässt vermuten, dass die Trainingsstabilität des Attention-Mechanismus stark von den Eigenschaften des Datensatzes abhängt – etwa der Sequenzlänge, der Variablenanzahl oder dem Störverhalten. Studien zeigen, dass Attention-Mechanismen zwar die Leistung steigern können, dabei aber oft zu höherer Trainingsinstabilität führen, insbesondere bei komplexen und verrauschten Daten [55].

Zusammenfassend zeigen alle Modelle spezifische Schwächen: Das BiLSTM-VAE-Modell zeigt stark datensatzabhängige Stabilität bei gleichzeitig insgesamt schwacher Leistung, das BiLSTM-AE-Modell ist teils instabil, aber verlässlich, und das Attention-based-BiLSTM-AE-Modell ist leistungsstark, jedoch nicht in allen Datensätzen gleich stabil. Diese Aspekte müssen bei einer praktischen Anwendung in Bezug auf Robustheit, Reproduzierbarkeit und Interpretierbarkeit kritisch berücksichtigt werden.

8.3 Ausblick

Die Ergebnisse dieser Arbeit zeigen, dass rekurrente Autoencoder-Architekturen mit Attention-Mechanismus grundsätzlich in der Lage sind, Anomalien in industriellen Zeitreihendaten zuverlässig zu erkennen. Dabei wurde deutlich, dass das Verhalten der Modelle stark vom jeweiligen Datensatz abhängt. Während das Attention-based BiLSTM-AE-Modell im TEP-Datensatz zwar die besten durchschnittlichen Ergebnisse erzielte, wies es dort auch eine hohe Varianz zwischen den Trainingsläufen auf. Im Gegensatz dazu

zeigte dasselbe Modell im Filling-Station-Datensatz eine vergleichsweise konstante und stabile Leistung. Umgekehrt verhielt es sich beim BiLSTM-VAE-Modell, das im TEP-Datensatz äußerst stabile, jedoch schwache Ergebnisse lieferte, im Filling-Station-Datensatz dagegen eine stark erhöhte Varianz aufwies. Diese Beobachtungen verdeutlichen, dass die Modellstabilität und -leistung stark von den Charakteristika der Eingangsdaten beeinflusst werden.

Auf Basis dieser Erkenntnisse ergeben sich verschiedene potenzielle Weiterentwicklungen. Ein naheliegender Ansatz wäre der Einsatz von Transformer-basierten Architekturen, die vollständig auf rekurrente Strukturen verzichten und durch reine Attention-Mechanismen eine globale Modellierung zeitlicher Abhängigkeiten ermöglichen. Studien, unter anderem die Masterarbeit von Schoch [7], zeigen, dass Transformer-Modelle auf dem TEP-Datensatz vielversprechende Ergebnisse erzielen können und gleichzeitig eine höhere Stabilität aufweisen.

Zukünftige Arbeiten könnten darüber hinaus gezielt die aufgezeigten Schwächen des BiLSTM-VAE-Ansatzes adressieren. Dazu zählen unter anderem eine verbesserte Gestaltung des latenten Raums sowie alternative Regularisierungsmethoden zur besseren Trennung zwischen Normal- und Anomaliedaten. Auch der Einsatz erklärbarer Modelle oder kombinierter Ansätze könnte helfen, die Interpretierbarkeit zu erhöhen und die praktischen Einsatzmöglichkeiten weiter zu verbessern.

Ebenso wäre eine systematische Hyperparameteroptimierung sowie die Erweiterung um Cross-Validation oder zusätzliche Datensätze sinnvoll, um die Generalisierbarkeit der Modelle zu verbessern. Die kontrastierenden Ergebnisse zwischen TEP und Filling Station zeigen deutlich, dass zukünftige Methoden robuster gegenüber variierenden Datenstrukturen sein sollten. Langfristig könnten solche Ansätze dazu beitragen, robuste und praxisnahe Anomalieerkennungssysteme für industrielle Anwendungen zu realisieren.

Literatur

- [1] R. Chalapathy und S. Chawla, “Deep Learning for Anomaly Detection: A Survey”, *arXiv preprint arXiv:1901.03407*, 2019. arXiv: 1901.03407. Adresse: <https://arxiv.org/abs/1901.03407>.
- [2] L. Ruff, R. A. Vandermeulen, N. Görnitz, A. Binder, E. Müller und M. Kloft, “A Unifying Review of Deep and Shallow Anomaly Detection”, *Proceedings of the IEEE*, Jg. 109, Nr. 5, S. 756–795, 2021. DOI: 10.1109/JPROC.2021.3052449.
- [3] R. Chalapathy, E. T. T. Tan und S. Chawla, “Anomaly Detection Using One-Class Neural Networks”, *arXiv preprint arXiv:1802.06360*, 2018.
- [4] S. Hochreiter und J. Schmidhuber, “Long Short-Term Memory”, *Neural Computation*, Jg. 9, Nr. 8, S. 1735–1780, 1997.
- [5] D. P. Kingma und M. Welling, “Auto-Encoding Variational Bayes”, *arXiv preprint arXiv:1312.6114*, 2014.
- [6] A. Vaswani, N. Shazeer, N. Parmar u. a., “Attention Is All You Need”, S. 5998–6008, 2017.
- [7] F. Schoch, *Anomalieerkennung in industriellen Produktionsprozessen mit Transformer-Modellen*, 2024.
- [8] V. Chandola, A. Banerjee und V. Kumar, “Anomaly Detection: A Survey”, *ACM Computing Surveys*, Jg. 41, Nr. 3, 15:1–15:58, 2009. DOI: 10.1145/1541880.1541882.
- [9] M. Gupta, J. Gao, C. Aggarwal und J. Han, “Outlier Detection for Temporal Data: A Survey”, *IEEE Transactions on Knowledge and Data Engineering*, Jg. 26, Nr. 9, S. 2250–2267, 2014. DOI: 10.1109/TKDE.2013.184.
- [10] M. Goldstein und S. Uchida, “A Comparative Evaluation of Unsupervised Anomaly Detection Algorithms for Multivariate Data”, *PLOS ONE*, Jg. 11, Nr. 4, e0152173, 2016. DOI: 10.1371/journal.pone.0152173.
- [11] M. Markou und S. Singh, “Novelty detection: a review—part 1: statistical approaches”, *Signal Processing*, Jg. 83, Nr. 12, S. 2481–2497, 2003. DOI: 10.1016/j.sigpro.2003.07.018.
- [12] M. Markou und S. Singh, “Novelty detection: a review—part 2: neural network-based approaches”, *Signal Processing*, Jg. 83, Nr. 12, S. 2499–2521, 2003. DOI: 10.1016/j.sigpro.2003.07.019.

- [13] F. T. Liu, K. M. Ting und Z.-H. Zhou, “Isolation Forest”, in *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, IEEE, 2008, S. 413–422. DOI: 10.1109/ICDM.2008.17.
- [14] C. Zhou und R. C. Paffenroth, “Anomaly Detection with Robust Deep Autoencoders”, *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, S. 665–674, 2017. DOI: 10.1145/3097983.3098052.
- [15] P. Malhotra, L. Vig, G. Shroff und P. Agarwal, “Long Short Term Memory Networks for Anomaly Detection in Time Series”, *European Symposium on Artificial Neural Networks (ESANN)*, 2015. arXiv: 1508.01195.
- [16] J. Herzen, *Darts: Time Series Made Easy in Python*, Accessed: March 10, 2025, 2021. Adresse: <https://unit8.com/resources/darts-time-series-made-easy-in-python/>.
- [17] Y. Ni, F. C. Stingo und V. Baladandayuthapani, “Bayesian Covariate-Dependent Gaussian Graphical Models with Varying Structure”, *Journal of Machine Learning Research*, Jg. 23, S. 1–29, 2022. Adresse: <http://jmlr.org/papers/v23/21-0102.html>.
- [18] I. Goodfellow, Y. Bengio und A. Courville, *Deep Learning*. MIT Press, 2016. Adresse: <https://www.deeplearningbook.org/>.
- [19] G. E. Hinton und R. R. Salakhutdinov, “Reducing the Dimensionality of Data with Neural Networks”, *Science*, Jg. 313, Nr. 5786, S. 504–507, 2006. DOI: 10.1126/science.1127647.
- [20] Y. Bengio, “Learning Deep Architectures for AI”, *Foundations and Trends in Machine Learning*, Jg. 2, Nr. 1, S. 1–127, 2009. DOI: 10.1561/22000000006.
- [21] P. Vincent, H. Larochelle, Y. Bengio und P.-A. Manzagol, “Extracting and Composing Robust Features with Denoising Autoencoders”, in *Proceedings of the 25th International Conference on Machine Learning (ICML)*, 2008, S. 1096–1103. DOI: 10.1145/1390156.1390294.
- [22] D. P. Kingma und M. Welling, *Auto-Encoding Variational Bayes*, 2022. arXiv: 1312.6114 [stat.ML]. Adresse: <https://arxiv.org/abs/1312.6114>.
- [23] M. I. Jordan, Z. Ghahramani, T. S. Jaakkola und L. K. Saul, “An Introduction to Variational Methods for Graphical Models”, *Machine Learning*, 1999. DOI: 10.1023/A:1007665907178.
- [24] D. E. Rumelhart, G. E. Hinton und R. J. Williams, “Learning representations by back-propagating errors”, *Nature*, Jg. 323, S. 533–536, 1986. DOI: 10.1038/323533a0.

- [25] J. L. Elman, “Finding Structure in Time”, *Cognitive Science*, Jg. 14, Nr. 2, S. 179–211, 1990. DOI: 10.1207/s15516709cog1402_1.
- [26] Y. Bengio, P. Simard und P. Frasconi, “Learning Long-Term Dependencies with Gradient Descent is Difficult”, *IEEE Transactions on Neural Networks*, Jg. 5, Nr. 2, S. 157–166, 1994. DOI: 10.1109/72.279181.
- [27] S. Hochreiter und J. Schmidhuber, “Long Short-Term Memory”, *Neural Computation*, Jg. 9, Nr. 8, S. 1735–1780, 1997. DOI: 10.1162/neco.1997.9.8.1735.
- [28] X.-C. Le, T.-T. Nguyen und N.-X. Van, “An LSTM neural network model for predicting flood in Vietnam”, *Journal of Hydrology*, Jg. 573, S. 602–611, 2019. DOI: 10.1016/j.jhydrol.2019.03.093.
- [29] C. Olah, *Understanding LSTMs*, 2015. Adresse: <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>.
- [30] A. Graves und J. Schmidhuber, “Framewise phoneme classification with bidirectional LSTM and other neural network architectures”, *Neural Networks*, Jg. 18, Nr. 5-6, S. 602–610, 2005. DOI: 10.1016/j.neunet.2005.06.042.
- [31] A. Mishra, A. Sharma und I. Bhattacharya, “Attention-Based Bi-LSTM for Anomaly Detection on Time-Series Data”, *ResearchGate*, 2021. Adresse: https://www.researchgate.net/publication/354525355_Attention-Based_Bi-LSTM_for_Anomaly_Detection_on_Time-Series_Data.
- [32] D. Lee, “Revolutionizing Time Series Prediction with LSTM and the Attention Mechanism”, *Towards Data Science*, 2022. Adresse: <https://drlee.io/revolutionizing-time-series-prediction-with-lstm-with-the-attention-mechanism-090833a19af9>.
- [33] Z. Lin, M. Feng, C. N. Santos u. a., “A Structured Self-Attentive Sentence Embedding”, *arXiv preprint arXiv:1703.03130*, 2017. arXiv: 1703.03130.
- [34] M. Raihan und A. Ahmed, “Enhancing Anomaly Detection in Windfarm Time-Series Using BiLSTM-AE”, *International Journal of Energy Research*, Jg. 47, Nr. 5, S. 1234–1245, 2023.
- [35] A. Staffini, H. J. Lee und D. Park, “A Disentangled VAE-BiLSTM Model for Heart Rate Anomaly Detection”, *IEEE Transactions on Neural Networks and Learning Systems*, Jg. 34, Nr. 6, S. 2071–2082, 2023.
- [36] J. Kim, S. Park und H. Yoo, “Attention-based BiLSTM Autoencoder for Time-Series Anomaly Detection”, *Journal of the Korea Society of Computer and Information*, Jg. 26, Nr. 5, S. 11–20, 2021. DOI: 10.9708/jksci.2021.26.05.011.

- [37] T. D. Beast, *Time Series Part 4: Determining the Window Size for Moving Averages*, Accessed: 2024-03-14, 2023. Adresse: <https://medium.com/@thedatabeast/time-series-part-4-determining-the-window-size-for-moving-averages-a07c5cfcfac9>.
- [38] C. A. Rieth, B. D. Amsel, R. Tran und M. B. Cook, “Additional Tennessee Eastman Process Simulation Data for Anomaly Detection Evaluation”, Johns Hopkins University Applied Physics Laboratory, Techn. Ber., 2017. Adresse: <https://doi.org/10.48550/arXiv.1710.00086>.
- [39] J. Devlin, M.-W. Chang, K. Lee und K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”, *arXiv preprint arXiv:1810.04805*, 2018.
- [40] Y. Fu und F. Xue, “MAD: Self-supervised Masked Anomaly Detection Task for Multivariate Time Series”, in *2022 International Joint Conference on Neural Networks (IJCNN)*, IEEE, 2022, S. 1–8.
- [41] F. Institute, *Analyzing the Tennessee Eastman Process Dataset for Anomaly Detection*, 2022. Adresse: <https://medium.com/@victorspruela/lstm-autoencoders-detecting-anomalies-in-the-tennessee-eastman-process-6170b70a1d81>.
- [42] V. Spruela, *LSTM Autoencoders for Anomaly Detection in the Tennessee Eastman Process*, 2022. Adresse: <https://github.com/vicrsp/udacity-ds-2022/tree/main/src/capstone>.
- [43] M. Weiner und S. Müller, “Deep Anomaly Detection on Tennessee Eastman Process Data”, *Chemical Engineering and Processing*, Jg. 172, S. 104 345, 2022. DOI: 10.1002/cite.202200238.
- [44] M. Sakurada und T. Yairi, “Anomaly Detection Using Autoencoders with Nonlinear Dimensionality Reduction”, *Proceedings of the MLSDA 2014 2nd Workshop on Machine Learning for Sensory Data Analysis*, S. 4–11, 2014. DOI: 10.1145/2689746.2689747.
- [45] A. S. Raihan und I. Ahmed, “A Bi-LSTM Autoencoder Framework for Anomaly Detection – A Case Study of a Wind Power Dataset”, *arXiv preprint arXiv:2303.09703*, 2023. Adresse: <https://arxiv.org/abs/2303.09703>.
- [46] Y. Zhao, X. Zhang, Z. Shang und Z. Cao, “A Novel Hybrid Method for KPI Anomaly Detection Based on VAE and SVDD”, *Symmetry*, Jg. 13, S. 2104, Nov. 2021. DOI: 10.3390/sym13112104.

- [47] M. Matar, T. Xia, K. Huguenard, D. Huston und S. Wshah, *Multi-Head Attention based Bi-LSTM for Anomaly Detection in Multivariate Time-Series of WSN*, 2023. DOI: 10.1109/AICAS57966.2023.10168670.
- [48] J. An und S. Cho, “Variational Autoencoder based Anomaly Detection using Reconstruction Probability”, in *Special Interest Group on Knowledge Discovery and Data Mining (SIGKDD) Workshop on Anomaly Detection*, 2015, S. 1–8.
- [49] E. Park, H. J. Kim und K. M. Lee, “Multimodal Anomaly Detection for Unstructured and Structured Data”, *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, S. 2784–2792, 2018.
- [50] L. Correia, J.-C. Goos, P. Klein, T. Bäck und A. V. Kononova, “MA-VAE: Multi-head Attention-based Variational Autoencoder Approach for Anomaly Detection in Multivariate Time-series Applied to Automotive Endurance Powertrain Testing”, in *Proceedings of the 15th International Conference on Neural Computation Theory and Applications (NCTA)*, 2023, S. 407–418. DOI: 10.5220/0012163100003595. Adresse: <https://www.scitepress.org/Link.aspx?doi=10.5220/0012163100003595>.
- [51] F. Hartung, B. Franks, T. Michels u. a., “Deep Anomaly Detection on Tennessee Eastman Process Data”, English, *Chemie-Ingenieur-Technik*, Jg. 95, Nr. 7, S. 1077–1082, Juli 2023, Publisher Copyright: © 2023 The Authors. Chemie Ingenieur Technik published by Wiley-VCH GmbH., ISSN: 0009-286X. DOI: 10.1002/cite.202200238.
- [52] D. Bouman und T. Heskes, “Why autoencoders fail at anomaly detection: theoretical arguments and practical solutions”, *arXiv preprint arXiv:2401.06158*, 2024. Adresse: <https://arxiv.org/abs/2401.06158>.
- [53] J. Liu, Y. Zhang, F. Yang und Z. Li, “Fault Detection in Chemical Processes Using Variational Autoencoders with Temporal Priors”, *Industrial & Engineering Chemistry Research*, Jg. 61, Nr. 6, S. 2203–2214, 2022. DOI: 10.1021/acs.iecr.1c04179.
- [54] B. Zong, Q. Song, Y. Min u. a., “Deep autoencoding gaussian mixture model for unsupervised anomaly detection”, in *International Conference on Learning Representations (ICLR)*, 2018. Adresse: <https://openreview.net/forum?id=B1nNNzi0b>.
- [55] H. Zeng, W. Chen, M. Li und X. Gao, “Dual-Attention LSTM Autoencoder for Multivariate Time Series Anomaly Detection in Industrial Systems”, *IEEE Transactions on Industrial Informatics*, 2024. DOI: 10.1109/TII.2024.3364226.